```
// BootRunner.cs
using System.Collections;
using Sy.Core;
using UnityEngine;
using UnityEngine.SceneManagement;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyFarm.Boot
{
    public static class SceneIds
    {
        public const string Boot = "S00_Boot";
        public const string Main = "S13_Main";
    }
    /// &lt;summary&gt;
    /// S00_Boot scene host: register mock services -&gt; run seven steps -&gt; switch to main scene.
    /// On failure it stays on S00_Boot and logs the error (no dead-end while loops).
    /// &lt;/summary&gt;
    public sealed class BootRunner : MonoBehaviour
    {
        IEnumerator Start()
        {
            BootServices.RegisterAll();
            // B655: the boot beat carries the AIGC disclosure footer from the very
            // first rendered frame (first-frame face parity with the S13 scene face).
            FarmAbout.EnsureAigcLabel();
            var sequence = new BootSequence(SwitchToMain);
            if (!sequence.RunAll())
            {
                Logger.Error("BOOT", "boot sequence failed - stay on S00_Boot");
                yield break;
            }
            yield return null;
        }
        bool SwitchToMain()
        {
            StartCoroutine(LoadMainRoutine());
            return true;
        }
        IEnumerator LoadMainRoutine()
        {
            var op = SceneManager.LoadSceneAsync(SceneIds.Main, LoadSceneMode.Single);
            while (op != null &amp;&amp; !op.isDone) yield return null;
        }
    }
}
// BootSequence.cs
using System;
using System.Collections.Generic;
using Sy.Core;
using Sy.Platform;
```

```
namespace Sy.CrazyFarm.Boot
{
    /// &lt;summary&gt;
    /// Seven-step boot sequence: Logger -&gt; Time -&gt; Config -&gt; Save -&gt; Locale -&gt; Ads.Preload -&gt; SwitchM
    /// each step logs "[BOOT] n/7 name ok|FAIL". A step exception never blocks later steps.
    /// &lt;/summary&gt;
    public sealed class BootSequence
    {
        public const int TotalSteps = 7;
        readonly Func&lt;bool&gt; _switchScene;
        readonly List&lt;string&gt; _lines = new();
        public BootSequence(Func&lt;bool&gt; switchScene)
        {
            _switchScene = switchScene;
        }
        public IReadOnlyList&lt;string&gt; Lines =&gt; _lines;
        public bool RunAll()
        {
            bool ok = true;
            ok &amp;= Run(1, "Logger", LoggerStep);
            ok &amp;= Run(2, "Time", TimeStep);
            ok &amp;= Run(3, "Config", ConfigStep);
            ok &amp;= Run(4, "Save", SaveStep);
            ok &amp;= Run(5, "ILocale", LocaleStep);
            ok &amp;= Run(6, "Ads.Preload", AdsStep);
            ok &amp;= Run(7, "SwitchMain", () =&gt; _switchScene != null &amp;&amp; _switchScene());
            return ok;
        }
        bool Run(int n, string name, Func&lt;bool&gt; step)
        {
            bool result;
            try
            {
                result = step();
            }
            catch (Exception e)
            {
                Logger.Error("BOOT", $"step {n} {name} exception: {e.Message}");
                result = false;
            }
            string line = $"[BOOT] {n}/{TotalSteps} {name} {(result ? "ok" : "FAIL")}";
            _lines.Add(line);
            Logger.Info("BOOT", line);
            return result;
        }
        static bool LoggerStep() =&gt; true; // static facade self-initializes on first call
        static bool TimeStep()
        {
            var time = ServiceLocator.Get&lt;ITimeService&gt;();
            return time.ServerNowMs() &gt; 0 &amp;&amp; !time.TimeCheated();
```

```
        }
        static bool ConfigStep()
        {
            var config = ServiceLocator.Get&lt;IConfigService&gt;();
            var update = config.CheckUpdate("mock://manifest");
            return update.NewVersion != null; // mock = no hot update
        }
        static bool SaveStep()
        {
            var save = ServiceLocator.Get&lt;ISaveService&gt;();
            save.Save("boot_probe", "{}");
            return !string.IsNullOrEmpty(save.Load("boot_probe"));
        }
        static bool LocaleStep()
        {
            var locale = ServiceLocator.Get&lt;ILocaleService&gt;();
            return !string.IsNullOrEmpty(locale.GetFontSet().NormalFont);
        }
        static bool AdsStep()
        {
            ServiceLocator.Get&lt;IAdsService&gt;().Preload("boot_warmup");
            return true;
        }
    }
}
// BootServices.cs
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Core;
using Sy.Platform;
using UnityEngine;
using Logger = Sy.Core.Logger;
namespace Sy.CrazyFarm.Boot
{
    /// &lt;summary&gt;
    /// Dev-time mock service set (SY_MOCK scope). All seven boot steps depend on this set.
    /// T010: the ads service rides the channel suite (PlatformBootstrap) so
    /// ShowRewarded consumption is category-capped on the shared AdCap in
    /// every channel - the offline doubler pre-check and the consume can
    /// never disagree.
    /// &lt;/summary&gt;
    public static class BootServices
    {
        public static readonly AdCapPolicy AdCap = new AdCapPolicy();
        /// &lt;summary&gt;Resolved channel suite (T010). Non-null once RegisterAll
        /// ran; the suite's ads service is what ServiceLocator hands out.&lt;/summary&gt;
        public static PlatformSuite Suite { get; private set; }
        public static void RegisterAll()
        {
```

```
        if (ServiceLocator.TryGet&lt;ITimeService&gt;(out _)) return; // idempotent: double register would throw
        ServiceLocator.Register&lt;ITimeService&gt;(new MockTimeService());
        ServiceLocator.Register&lt;IConfigService&gt;(new MockConfigService());
        ServiceLocator.Register&lt;ISaveService&gt;(new MockSaveService());
        ServiceLocator.Register&lt;ILocaleService&gt;(new MockLocaleService());
        // T010: channel suite wiring. PlatformBootstrap gates SY_WECHAT /
        // SY_DOUYIN builds to the real channel suites; editor/batchmode stays
        // mock. Ads consume from the shared AdCap in every channel so the
        // offline-doubler pre-check/consume same-instance law survives the
        // suite switch.
        Suite = PlatformBootstrap.Create(AdCap);
        ServiceLocator.Register&lt;PlatformSuite&gt;(Suite);
        ServiceLocator.Register&lt;IAdsService&gt;(Suite.Ads);
        Logger.Info("BOOT", "services registered (suite=" + PlatformBootstrap.SuiteIdFor(Suite.Channel) + ")");
    }
}
public sealed class MockTimeService : ITimeService
{
    public TimeSource Source =&gt; TimeSource.Mock;
    public long ClientNowMs() =&gt; DateTimeOffset.UtcNow.ToUnixTimeMilliseconds();
    /// &lt;summary&gt;SY_MOCK: device time doubles as server time until the real chain lands.&lt;/summary&gt;
    public long ServerNowMs() =&gt; ClientNowMs();
    public bool TimeCheated() =&gt; Math.Abs(ServerNowMs() - ClientNowMs()) &gt; 120_000;
}
public sealed class MockConfigService : IConfigService
{
    public T GetTable&lt;T&gt;(string tableId) where T : class
        =&gt; throw new KeyNotFoundException("mock config table not loaded: " + tableId);
    public bool TryGetTable&lt;T&gt;(string tableId, out T table) where T : class
    {
        table = null;
        return false;
    }
    public ConfigUpdateResult CheckUpdate(string manifestUrl)
        =&gt; new ConfigUpdateResult(false, "mock-0.0.1");
    public void SwapPolicy(ConfigSwapPolicy policy) { }
    public string PinConfigVersion(string scope) =&gt; "mock-0.0.1";
}
public sealed class MockSaveService : ISaveService
{
    const string Prefix = "sy_save_";
    public void Save(string slot, string payloadJson) =&gt; PlayerPrefs.SetString(Prefix + slot, payloadJson);
    public string Load(string slot) =&gt; PlayerPrefs.GetString(Prefix + slot, string.Empty);
    /// &lt;summary&gt;No cloud: local storage is the authority (zero-server doctrine).&lt;/summary&gt;
    public SyncResult CloudSync() =&gt; new SyncResult(false, null);
}
public sealed class MockLocaleService : ILocaleService
{
    public void SetLang(string lang) { }
    /// &lt;summary&gt;Missing key falls back to the key itself with a warning.&lt;/summary&gt;
```

```
        public string GetText(string key, params string[] args)
        {
            Logger.Warn("LOCALE", "missing i18n key (fallback): " + key);
            return key;
        }
        // U184 real-device CJK face (B569 chain step 3, B655 wiring): LegacyRuntime.ttf
        // ships no zh glyphs, so the zh-CN display lines render blank on real
        // devices. The B569 fleet-cast NotoSansSC subset (201KB, cmap-verified
        // missing=0 over the fleet union incl. the CrazyFarm 302-char census)
        // rides Resources as NotoSansSC-CrazySubset.otf; future view faces
        // resolve this name and degrade to LegacyRuntime on a miss.
        public FontSet GetFontSet()
            =&gt; new FontSet("NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf", "NotoSansSC-CrazySubset.otf");
    }
}
// FarmAbout.cs
// FarmAbout.cs - the AIGC disclosure footer builder (G13, B655).
// FE-20260924-B-11 / E-071: the eleven TJ AI-assisted art pieces in the
// catalog (B406 wave) plus the wired AI-generated BGM (g13_bgm_main, B582)
// flip the 2025-09-01 AI-content label rules on for this game - so the duty
// line covers BOTH faces from day one (the G04 B654 paradigm re-cut for
// G13's scene shell; the G04 art-only first cut does not apply here because
// the audio leg is already wired). The label is a persistent ROOT-LEVEL
// footer - not a Canvas child - so the frozen scene shell (Canvas, hosts,
// EventSystem, camera set) never sees it; camera-relative, parked in the
// 9x16 design frame's bottom strip below the DeviceMatrix fixture union
// floor (-7.70), frame bottom -8.00. Unity nulls the instance on scene
// switches so every scene face (S00 boot beat, S13 main) rebuilds its own
// footer through the same helper.
using UnityEngine;
namespace Sy.CrazyFarm
{
    public static class FarmAbout
    {
        /// &lt;summary&gt;The explicit AI-content mark (art + audio single version;
        /// ASCII-only, zero CJK - the LegacyRuntime face renders it clean).&lt;/summary&gt;
        public const string AigcLineText = "VISUAL ART AND AUDIO ASSETS IN THIS GAME ARE AI-ASSISTED";
        /// &lt;summary&gt;Live footer instance (root object; idempotent; Unity nulls
        /// it on scene switches so every scene rebuilds its own).&lt;/summary&gt;
        public static GameObject AigcLabel;
        /// &lt;summary&gt;Builds (or returns) the persistent AIGC disclosure footer.
        /// Called from the S13 main-scene face (FarmCameraFitHost, after the
        /// camera fit so the camera-relative placement lands on the fitted
        /// design frame) and the S00 boot beat (BootRunner) - first-frame
        /// face parity.&lt;/summary&gt;
        public static GameObject EnsureAigcLabel()
        {
            if (AigcLabel != null) return AigcLabel;
            var go = new GameObject("AigcLabel");
            Camera cam = Camera.main;
```

```
            Vector3 camPos = cam != null ? cam.transform.position : Vector3.zero;
            go.transform.position = new Vector3(camPos.x, camPos.y - 7.85f, -1f);
            go.transform.localScale = Vector3.one * 0.1f;
            var tm = go.AddComponent&lt;TextMesh&gt;();
            tm.font = Resources.GetBuiltinResource&lt;Font&gt;("LegacyRuntime.ttf");
            tm.text = AigcLineText;
            tm.fontSize = 16;
            tm.anchor = TextAnchor.MiddleCenter;
            tm.color = new Color(0.78f, 0.80f, 0.83f, 1f);   // quiet steel: legible on art and wash alike
            AigcLabel = go;
            return go;
        }
    }
}
// FarmAlbum.cs
// FarmAlbum.cs - the album aggregator, the honor wall, the harvest war
// report and the second-run carry (G13-T011a, SYS08/M10).
// A pure read-only aggregation over the frozen card faces: the three
// album tabs (crops / livestock / thieves), the four-season honor wall
// (one golden-wheat badge per completed volume), the hand-drawn harvest
// war report - the category-unique flex card: the header, the season and
// game day, the three volume count lines, the gold ledger, the latest
// signature crop and the farmer's sign-off - and the second-run
// inheritance face (the 20% gold floor, the full album snapshot, the
// village ties kept, the rank restarting at the one-mu plot). Zero new
// game state and zero changes on FarmCards/FarmStage: every face reads
// the live engines (the album law gates itself through collection). The
// share tap books its three-line funnel in order (tap -&gt; card composed -&gt;
// landed - the local zero-server share face, red line 12). The empire
// ending stands armed at rank &gt;= R5 (the panorama-light show is the view
// slice's consumption). Pure logic, no MonoBehaviour, zero RNG: the same
// state always fingerprints byte-identical.
using System;
using System.Collections.Generic;
namespace Sy.CrazyFarm
{
    /// &lt;summary&gt;One album tab face: the volume id, the owned ids (the
    /// collection order mirrors FarmCards.Owned), the frozen volume size
    /// and the completion flag - a pure query, zero new state.&lt;/summary&gt;
    public sealed class AlbumVolumeFace
    {
        public string id = "";
        public string[] ownedIds = Array.Empty&lt;string&gt;();
        public int size;
        public bool complete;
    }
    /// &lt;summary&gt;The honor wall face: the completed-volume count, the
    /// four-season panorama with the live season and game day, and the
    /// golden-wheat badges - one per completed volume.&lt;/summary&gt;
    public sealed class HonorWallFace
```

```
{
    public int completedVolumes;
    public string[] seasons = Array.Empty&lt;string&gt;();   // the four-season panorama
    public string seasonId = "";                        // the live season
    public int gameDay;
    public int goldenWheatBadges;
}
/// &lt;summary&gt;The second-run carry (M10): the 20% gold floor, the full
/// album snapshot, the village ties kept and the rank restart. A pure
/// function face - the new run consumes it, nothing ever mutates for
/// it.&lt;/summary&gt;
public sealed class NewRunInheritanceFace
{
    public int inheritGoldHc;
    public string[] albumsKept = Array.Empty&lt;string&gt;();
    public bool tiesKept;
    public bool rankReset;
}
public sealed class FarmAlbum
{
    readonly FarmStage _stage;
    readonly FarmCards _cards;
    public FarmAlbum(FarmStage stage, FarmCards cards = null)
    {
        _stage = stage;
        _cards = cards ?? stage.Cards;
    }
    // ---- face 1: the three album tabs (pure query, owned read-only) ----
    /// &lt;summary&gt;The three frozen tab ids (crops / livestock /
    /// thieves).&lt;/summary&gt;
    public static string[] TabIds() =&gt; FarmCards.AlbumVolumeIds();
    /// &lt;summary&gt;One tab face over the frozen volume: the owned ids
    /// mirror FarmCards.OwnedInVolume exactly (the card system stays
    /// the single collection authority - zero new state here, the
    /// T011b gate reconciles the two faces against each other).&lt;/summary&gt;
    public AlbumVolumeFace VolumeFace(string albumId)
    {
        var owned = new List&lt;string&gt;();
        foreach (var id in _cards.Owned)
            if (AlbumOf(id) == albumId) owned.Add(id);
        var face = new AlbumVolumeFace
        {
            id = albumId,
            ownedIds = owned.ToArray(),
            size = _cards.VolumeSize(albumId),
        };
        face.complete = face.ownedIds.Length &gt;= face.size;
        return face;
    }
    /// &lt;summary&gt;The card album mapping, mirrored through the public
```

```
/// roster (the exact law FarmCards.OwnedInVolume rides).&lt;/summary&gt;
static string AlbumOf(string cardId)
{
    foreach (var c in FarmCards.Roster()) if (c.id == cardId) return c.albumId;
    return "";
}
// ---- face 2: the four-season honor wall ----
/// &lt;summary&gt;The honor wall: the completed volumes, the season
/// panorama with the live season and game day, and one
/// golden-wheat badge per completed volume.&lt;/summary&gt;
public HonorWallFace HonorWall()
{
    int completed = _cards.CompletedVolumes();
    return new HonorWallFace
    {
        completedVolumes = completed,
        seasons = (string[])FarmData.SeasonIds.Clone(),
        seasonId = _stage.CurrentSeasonId,
        gameDay = _stage.GameDay,
        goldenWheatBadges = completed,
    };
}
// ---- face 3: the harvest war report (the flex card) ----
/// &lt;summary&gt;The signature crop: the latest unlocked crop card (the
/// collection order is the unlock order). Empty when no crop has
/// joined the album yet.&lt;/summary&gt;
public string SignatureCropId
{
    get
    {
        var owned = _cards.Owned;
        for (int i = owned.Count - 1; i &gt;= 0; i--)
            if (AlbumOf(owned[i]) == "album_crop") return owned[i];
        return "";
    }
}
/// &lt;summary&gt;The hand-drawn war-report payload: the header, the
/// season and game day, the three volume counts, the gold ledger,
/// the signature crop and the sign-off - composed from the freshest
/// state, byte-identical for the same state (the FarmOffline
/// ComposeSummary law). Text keys ride FarmTexts with En/zhCN
/// parity.&lt;/summary&gt;
public string ComposeReportCard()
{
    var crop = VolumeFace("album_crop");
    var livestock = VolumeFace("album_livestock");
    var thief = VolumeFace("album_thief");
    var sb = new System.Text.StringBuilder();
    sb.Append(FarmTexts.EnText(FarmTexts.KeyReportHeader));
    sb.Append('\n').Append(string.Format(FarmTexts.EnText(FarmTexts.KeyReportSeason),
```

```
            SeasonNameOf(_stage.CurrentSeasonId), _stage.GameDay));
        sb.Append('\n').Append(string.Format(FarmTexts.EnText(FarmTexts.KeyReportAlbums),
            crop.ownedIds.Length, crop.size,
            livestock.ownedIds.Length, livestock.size,
            thief.ownedIds.Length, thief.size));
        sb.Append('\n').Append(string.Format(FarmTexts.EnText(FarmTexts.KeyReportGold),
            _stage.Growth.GoldHc / 100));
        var sig = SignatureCropId;
        sb.Append('\n').Append(string.Format(FarmTexts.EnText(FarmTexts.KeyReportSignature),
            sig == "" ? "-" : CropNameOf(sig)));
        sb.Append('\n').Append(FarmTexts.EnText(FarmTexts.KeyReportFooter));
        return sb.ToString();
    }
    /// &lt;summary&gt;The share tap: books the three-line funnel in order
    /// (tap -&gt; card composed -&gt; landed) and returns the composed card.
    /// The share itself stays local (red line 12 - zero server
    /// faces).&lt;/summary&gt;
    public string TapShare()
    {
        Funnel.Add("evt_album_share_tap@" + Stamp());
        var card = ComposeReportCard();
        Funnel.Add("evt_album_share_card@" + Stamp());
        Funnel.Add("evt_album_share_landed@" + Stamp());
        return card;
    }
    /// &lt;summary&gt;evt_album_share funnel (T011a): the three share lines
    /// land in order for the T011b gate assertions.&lt;/summary&gt;
    public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
    string Stamp() =&gt; _stage.SimTime.ToString("0.00");
    // ---- face 4: the second-run inheritance (M10, pure function) ----
    /// &lt;summary&gt;The next-farmer carry: the frozen 20% gold floor, the
    /// full album snapshot (owned ids keep their order), the village
    /// ties kept and the rank restarting at the one-mu plot. Reads the
    /// frozen constants - a pure function of its inputs.&lt;/summary&gt;
    public static NewRunInheritanceFace NewRunInheritance(int goldHc, FarmCards cards)
    {
        var owned = new List&lt;string&gt;();
        if (cards != null)
            foreach (var id in cards.Owned) owned.Add(id);
        return new NewRunInheritanceFace
        {
            inheritGoldHc = goldHc * FarmData.InheritGoldPercent / 100,
            albumsKept = owned.ToArray(),
            tiesKept = FarmData.InheritVillagerTies,
            rankReset = FarmData.InheritRankReset,
        };
    }
    // ---- face 5: the empire ending (armed face) ----
    /// &lt;summary&gt;True once the ladder tops out at R5: the panorama
    /// light show stands armed (the view slice consumes the face -
```

```
        /// the all-lit ending beat).&lt;/summary&gt;
        public bool EmpireEndingArmed =&gt; _stage.Growth.Rank.rank &gt;= FarmData.RankCount;
        // ---- the copy mirrors (compiled copy surface, zero restated ids) ----
        static readonly string[] SeasonTextKeys =
        {
            FarmTexts.KeySeasonSpring, FarmTexts.KeySeasonSummer,
            FarmTexts.KeySeasonAutumn, FarmTexts.KeySeasonWinter,
        };
        static readonly string[] CropTextKeys =
        {
            FarmTexts.KeyCropLoveRadish, FarmTexts.KeyCropCrookedCorn, FarmTexts.KeyCropShyMushroom,
            FarmTexts.KeyCropTwinWatermelon, FarmTexts.KeyCropSnoringPumpkin, FarmTexts.KeyCropIntrovertSunflower,
        };
        static string SeasonNameOf(string seasonId)
        {
            var ids = FarmData.SeasonIds;
            for (int i = 0; i &lt; ids.Length &amp;&amp; i &lt; SeasonTextKeys.Length; i++)
                if (ids[i] == seasonId) return FarmTexts.EnText(SeasonTextKeys[i]);
            return seasonId;
        }
        static string CropNameOf(string cropId)
        {
            var crops = FarmData.Crops();
            for (int i = 0; i &lt; crops.Length &amp;&amp; i &lt; CropTextKeys.Length; i++)
                if (crops[i].id == cropId) return FarmTexts.EnText(CropTextKeys[i]);
            return cropId;
        }
        /// &lt;summary&gt;Canonical fingerprint: the zero-RNG parity instrument
        /// (the same state = the same bytes - the T011b freeze face
        /// consumes it).&lt;/summary&gt;
        public string Fingerprint()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("vols=").Append(_cards.CompletedVolumes());
            sb.Append("|season=").Append(_stage.CurrentSeasonId).Append(',').Append(_stage.GameDay);
            sb.Append("|sig=").Append(SignatureCropId);
            sb.Append("|report=").Append(ComposeReportCard().Length);
            sb.Append("|ending=").Append(EmpireEndingArmed ? 1 : 0);
            sb.Append("|funnel=").Append(Funnel.Count);
            return sb.ToString();
        }
    }
}
// FarmAlbumCard.cs
// FarmAlbumCard.cs - the harvest war-report share card view (G13-T011b,
// SYS08/M10). The frozen T011a payload becomes the programmatic card: the
// hand-drawn war-report bytes project straight onto the shared
// ShareCardService spec - the header line the title, the season/albums/
// gold/signature lines the body, the farmer's sign-off the footer - and
// the shared service renders the RT card (the X033 four laws live there:
```

```
// the layer walk last, Canvas.ForceUpdateCanvases, the manual camera
// Render, the PNG readback) into persistentDataPath. Art-first (X027):
// the g13_sharecard stem ships in S2.5; today the procedural dark card
// carries the flex (a missing stem never breaks a view). Zero new copy
// and zero restated text: every line rides the frozen payload
// (byte-projection - the T011b gate reconciles the two faces against
// each other). The share itself stays local (red line 12 - zero server
// faces: a stable local query, never a round-trip). Pure view, no
// MonoBehaviour.
using System.Collections.Generic;
using Sy.Social;
namespace Sy.CrazyFarm
{
    public static class FarmAlbumCard
    {
        /// &lt;summary&gt;Art stem for the share-card template (the S2.5
        /// shipping seat; absent today = the procedural dark card face
        /// of the shared service).&lt;/summary&gt;
        public const string TemplateStem = "g13_sharecard";
        /// &lt;summary&gt;The local share query (red line 12: a stable local
        /// payload riding the platform share face, never a server).&lt;/summary&gt;
        public const string ShareQuery = "g13_harvest";
        /// &lt;summary&gt;The card spec projected from the frozen payload: the
        /// header line the title, the middle lines the body, the sign-off
        /// the footer - the card is a pure projection of
        /// ComposeReportCard (zero restated copy, byte-projection).&lt;/summary&gt;
        public static ShareCard SpecFrom(FarmAlbum album)
        {
            var spec = new ShareCard
            {
                TemplateKey = TemplateStem,
                Query = ShareQuery,
            };
            string[] lines = album.ComposeReportCard().Split('\n');
            if (lines.Length &gt; 0) spec.Title = lines[0];
            var body = new List&lt;string&gt;();
            for (int i = 1; i &lt; lines.Length - 1; i++) body.Add(lines[i]);
            spec.Lines = body.ToArray();
            if (lines.Length &gt; 1) spec.Footer = lines[lines.Length - 1];
            return spec;
        }
        /// &lt;summary&gt;Composes the PNG share card: the RT render and the PNG
        /// readback ride the shared ShareCardService (the tier ladder
        /// keeps the file inside the byte budget); the result carries
        /// Ok/Path/PngBytes/Width/Height - the T011b pixel canary consumes
        /// the written file. The share tap books its own three-line funnel
        /// on the album (TapShare); this face only paints.&lt;/summary&gt;
        public static ComposeResult Compose(FarmAlbum album)
        {
            var service = new ShareCardService();
```

```
            return service.Compose(SpecFrom(album));
        }
    }
}
// FarmAudioCatalog.cs
// FarmAudioCatalog.cs - generated-audio catalog (G13 audio wiring slice a).
// X027-isomorphic to the com.sy.core ArtCatalog (G11 WorkerAudioCatalog
// paradigm): the FarmAudioSetup editor tool scans Assets/_Game/Art/Audio and
// serializes clip references here so runtime code can resolve audio by
// filename stem. The asset lives at Assets/_Game/Resources/
// FarmAudioCatalog.asset (Resources keeps the playtest-slice lookup
// build-safe with zero Addressables wiring, same rationale as ArtCatalog).
// Missing clip = null = stay silent (X027 null-safe law): headless gates stay
// green whatever the import state is.
using System;
using System.Collections.Generic;
using UnityEngine;
namespace Sy.CrazyFarm
{
    [CreateAssetMenu(fileName = "FarmAudioCatalog", menuName = "Sy/CrazyFarm Audio Catalog")]
    public sealed class FarmAudioCatalog : ScriptableObject
    {
        [Serializable]
        public sealed class Entry
        {
            public string key;      // filename stem, e.g. "g13_bgm_main"
            public AudioClip clip; // direct reference keeps builds self-contained
        }
        public List&lt;Entry&gt; entries = new List&lt;Entry&gt;();
        Dictionary&lt;string, AudioClip&gt; _index;
        void OnEnable()
        {
            // Serializer revival can pair a fresh instance with a stale cache;
            // drop it so the next Get() rebuilds from entries.
            _index = null;
        }
        public AudioClip Get(string key)
        {
            if (string.IsNullOrEmpty(key)) return null;
            if (_index == null)
            {
                _index = new Dictionary&lt;string, AudioClip&gt;(entries.Count, StringComparer.Ordinal);
                for (int i = 0; i &lt; entries.Count; i++)
                {
                    var e = entries[i];
                    if (e != null &amp;&amp; !string.IsNullOrEmpty(e.key) &amp;&amp; e.clip != null &amp;&amp; !_index.C
                        _index[e.key] = e.clip;
                }
            }
            return _index.TryGetValue(key, out var c) ? c : null;
```

```
        }
    }
    /// &lt;summary&gt;
    /// Runtime lookup helper (ArtRuntime mirror). Consumers MUST treat null as
    /// "audio missing -&gt; stay silent" so headless checks stay green on any
    /// project state (catalog absent, clip pruned, name typo).
    /// &lt;/summary&gt;
    public static class FarmAudio
    {
        // Consumption stems (audio-book G13 section): the main BGM bed from the
        // B572 wave. Slice b wires the main-scene host; the constant is the
        // single consumption contract both sides code against.
        public const string BgmMain = "g13_bgm_main";
        static FarmAudioCatalog _catalog;
        public static FarmAudioCatalog Catalog
        {
            get
            {
                if (_catalog == null) _catalog = Resources.Load&lt;FarmAudioCatalog&gt;("FarmAudioCatalog");
                return _catalog;
            }
        }
        public static AudioClip Get(string key)
        {
            var c = Catalog;
            return c != null ? c.Get(key) : null;
        }
        /// &lt;summary&gt;
        /// Loop the requested stem on a fresh host object and return the source.
        /// Missing clip resolves to silence (returns null) - never an error, so
        /// the playtest slice runs with or without the audio wave landed.
        /// &lt;/summary&gt;
        public static AudioSource PlayLoop(string key, float volume = 0.6f)
        {
            var clip = Get(key);
            if (clip == null) return null;
            var go = new GameObject("FarmAudio(" + key + ")");
            var src = go.AddComponent&lt;AudioSource&gt;();
            src.clip = clip;
            src.loop = true;
            src.volume = volume;
            src.Play();
            return src;
        }
    }
}
// FarmAudioHost.cs
// FarmAudioHost.cs - S13_Main runtime hook for the generated-audio face
// (G13 audio wiring slice b). Mirror of FarmCameraFitHost: the scene-side
// face that starts the looping BGM bed on play start. FarmAudio.PlayLoop is
```

```
// null-safe by contract (missing catalog/clip = silence, never an error), so
// the headless gates stay green whatever the import state is.
using UnityEngine;
namespace Sy.CrazyFarm
{
    public sealed class FarmAudioHost : MonoBehaviour
    {
        void Start()
        {
            FarmAudio.PlayLoop(FarmAudio.BgmMain);
        }
    }
}
// FarmBoost.cs
// FarmBoost.cs - the season-window growth boost gate (G13-T015b, SYS10
// boost). The solar-term-window acceleration: the offer rides the Liqiu
// festival window face (the window opens = the natural recommendation
// slot - the LiveOps face and the IAA slot are the same piece), a tap
// prechecks the g13_boost 3/day category budget, the rewarded view
// consumes on the same policy instance and the same day bucket, and the
// completed view lands growth x2 for the current game day (the frozen
// festival x2 family, numeric pack section 7/10) - the stage day clock
// clears it on the next roll, so the window is exactly one game day.
// The three-piece law (G08 T009 / FarmOffline T010 precedent): the
// HasCategoryBudget precheck and the ShowRewarded consumption ride the
// same policy instance and the same day bucket (they can never
// disagree), the evt_ad funnel books offer -&gt; show -&gt; complete -&gt;
// boost_landed in order, and a capped tap refuses BEFORE the ad ever
// plays. In-window repeats refuse pre-offer with zero state change (the
// G11 wage-boost precedent). Pure logic, no MonoBehaviour, zero RNG.
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Sy.Platform;
namespace Sy.CrazyFarm
{
    public sealed class FarmBoost
    {
        /// &lt;summary&gt;Boost point id (T015b): draws from its own 3/day
        /// g13_boost safety-valve category (AdCategories.CategoryOfPoint
        /// authority, the helper-point precedent - premium-capped so it
        /// never farms).&lt;/summary&gt;
        public const string BoostPoint = "g13_boost";
        /// &lt;summary&gt;The frozen festival x2 family anchor (numeric pack
        /// section 7/10): the solar-term boost doubles the day's growth,
        /// the same x2 the Liqiu window's yield face rides.&lt;/summary&gt;
        public const int BoostFactorBp = 20000;
        readonly FarmStage _stage;
        readonly AdCapPolicy _adCap;
        readonly IAdsService _ads;
```

```
readonly Func&lt;string&gt; _dayKey;
public FarmBoost(FarmStage stage, AdCapPolicy adCap, IAdsService ads, Func&lt;string&gt; dayKey = null)
{
    _stage = stage;
    _adCap = adCap;
    _ads = ads;
    // the precheck reads the very bucket the registered service consumes
    // (injected key only for deterministic gates; production rides TodayKey)
    _dayKey = dayKey ?? AdCapPolicy.TodayKey;
    if (_ads != null) _ads.OnAdEvent += OnAdLifecycle;
}
/// &lt;summary&gt;Window face: the festival window owns the offer (the
/// window opens = the natural recommendation slot; the window is
/// one game day).&lt;/summary&gt;
public bool WindowActive =&gt; _stage != null &amp;&amp; _stage.Season.FestivalWindowActive;
/// &lt;summary&gt;Live boost-category precheck: always-fresh budget face
/// on the same policy instance and the same day bucket the
/// registered service consumes from, so precheck and consume can
/// never disagree.&lt;/summary&gt;
public bool OfferBudgetOk
    =&gt; _adCap != null
       &amp;&amp; _adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(BoostPoint));
/// &lt;summary&gt;Already landed today: repeat taps refuse pre-offer
/// with zero state change (the G11 wage-boost precedent).&lt;/summary&gt;
public bool BoostActiveToday =&gt; _stage != null &amp;&amp; _stage.Growth.BoostFactorBp != 10000;
/// &lt;summary&gt;The boost tap: window guard - already-landed guard -
/// budget precheck - rewarded view - the x2 lands for the game
/// day. A dead tap (window closed, boost active, or capped) burns
/// nothing: no offer entry, no ad, no budget.&lt;/summary&gt;
public async Task&lt;bool&gt; ClaimBoost()
{
    if (!WindowActive) return false;
    if (BoostActiveToday) return false;
    if (!OfferBudgetOk)
    {
        Funnel.Add("evt_ad_offer_capped@" + Stamp());   // the honest face refuses at the tap, never after the a
        return false;
    }
    Funnel.Add("evt_ad_offer@" + Stamp());
    RewardResult r = await _ads.ShowRewarded(BoostPoint);
    if (r == null || !r.Completed) return false;   // evt_ad_fail rides the lifecycle face; the retry window sta
    _stage.LandBoost(BoostFactorBp);   // the x2 lands on the growth engine + the stage bus on the same tick
    Funnel.Add("evt_boost_landed@" + Stamp());
    return true;
}
/// &lt;summary&gt;evt_ad funnel (T015b): lifecycle events of the boost
/// point land in the funnel list for gate assertions.&lt;/summary&gt;
void OnAdLifecycle(AdLifecycle e)
{
    if (e.PointId != BoostPoint) return;
```

```
            Funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + Stamp());
        }
        public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
        string Stamp() =&gt; _stage == null ? "0.00" : _stage.SimTime.ToString("0.00");
    }
}
// FarmCameraFit.cs
// FarmCameraFit.cs - deterministic device-adaptation contract (G13-T023, device matrix).
// Red line 10 portrait lock pairs with this fit: the farm column is a frozen
// DESIGN FRAME - the must-see panorama column of S09_Main (the gold/fame HUD
// band on top, the four-season farm field in the middle, the event-card and
// hint zone at the bottom, per the requirement doc S09_Main page) - that must
// stay fully visible inside the SAFE area on every device. The frame is a
// 9:16 portrait column (the 1080x1920 reference aspect), so base cards carry
// zero letterbox and notch/corner cards only ever letterbox, never clip.
// Apply computes an orthographic camera mapping the design frame into the
// safe rect; numbers come in as data so the headless gate drives tier cards
// (1080x1920 / 720x1280) deterministically; ApplyToMainCamera is the runtime
// face reading Screen (wired into S13_Main by ProjectSetup as
// FarmCameraFitHost). The published frame is the standing contract the
// upcoming panorama view slice must build inside - DeviceMatrixCheck is the
// drift tripwire.
using UnityEngine;
namespace Sy.CrazyFarm
{
    public static class FarmCameraFit
    {
        /// &lt;summary&gt;Design frame: the must-see farm column. 9 world units wide
        /// x 16 tall (9:16 = the portrait reference aspect, zero letterbox on
        /// a 1080x1920 base card). Top band = the gold/fame HUD panel; middle
        /// = the four-season panorama field with the plot slots; bottom band =
        /// the event-card zone + the hint row (S09_Main page contract).&lt;/summary&gt;
        public static readonly Rect DesignFrame = new Rect(-4.5f, -8.0f, 9.0f, 16.0f);
        /// &lt;summary&gt;Pixel-density floor per tier (px per world unit, safe-rect
        /// basis): a 0.5-wu UI tile must render at &gt;= 32 px on the worst card
        /// (720p + notch gives 72 px/wu, so the floor holds with headroom).&lt;/summary&gt;
        public const float PxPerWuFloor1080 = 100f;
        public const float PxPerWuFloor720 = 64f;
        /// &lt;summary&gt;Result of one fit: the safe rect in world space plus the
        /// pixels-per-world-unit the fit produced (legibility face).&lt;/summary&gt;
        public struct FitResult
        {
            public Rect SafeWorld;
            public float PxPerWu;
        }
        /// &lt;summary&gt;Fits the camera so the design frame sits fully inside the
        /// safe viewport rect. safeNv is normalized (x, y, w, h in 0..1).&lt;/summary&gt;
        public static FitResult Apply(Camera cam, int screenW, int screenH, Rect safeNv)
        {
            float safeWpx = screenW * safeNv.width;
```

```
            float safeHpx = screenH * safeNv.height;
            float safeAspect = safeWpx / safeHpx;
            float designAspect = DesignFrame.width / DesignFrame.height;
            // world units per pixel: bind the wider-relative axis, letterbox the other
            float wpp = designAspect &gt;= safeAspect
                ? DesignFrame.width / safeWpx
                : DesignFrame.height / safeHpx;
            cam.orthographic = true;
            cam.orthographicSize = screenH * 0.5f * wpp;
            float visW = screenW * wpp;
            float visH = screenH * wpp;
            // center the design frame on the safe rect's center
            float cu = safeNv.x + safeNv.width * 0.5f;
            float cv = safeNv.y + safeNv.height * 0.5f;
            float cx = DesignFrame.center.x - (cu - 0.5f) * visW;
            float cy = DesignFrame.center.y - (cv - 0.5f) * visH;
            cam.transform.position = new Vector3(cx, cy, cam.transform.position.z);
            float sx = cx + (safeNv.xMin - 0.5f) * visW;
            float sy = cy + (safeNv.yMin - 0.5f) * visH;
            return new FitResult
            {
                SafeWorld = new Rect(sx, sy, safeWpx * wpp, safeHpx * wpp),
                PxPerWu = 1f / wpp,
            };
        }
        /// &lt;summary&gt;Runtime face: fit the scene's main camera from the real
        /// screen and its safe area (notch/corners compensate by letterbox).
        /// Null-safe and skips on degenerate screen dims so headless gates
        /// that build compositions without a camera never break.&lt;/summary&gt;
        public static void ApplyToMainCamera()
        {
            Camera cam = Camera.main;
            if (cam == null) return;
            int w = Screen.width, h = Screen.height;
            if (w &lt;= 0 || h &lt;= 0) return;
            Rect safe = Screen.safeArea;
            Rect safeNv = new Rect(safe.x / w, safe.y / h, safe.width / w, safe.height / h);
            Apply(cam, w, h, safeNv);
        }
    }
}
// FarmCameraFitHost.cs
// FarmCameraFitHost.cs - S13_Main runtime hook for the device-adaptation
// contract (G13-T023). G13's scene shell has no game host yet (the panorama
// view slice lands with the S2.5 art batch), so this tiny component is the
// scene-side face that applies FarmCameraFit to the main camera on play
// start. Idempotent by construction: Apply is a pure function of the screen,
// so re-entering the scene refits the same numbers.
using UnityEngine;
namespace Sy.CrazyFarm
```

```
{
    public sealed class FarmCameraFitHost : MonoBehaviour
    {
        void Start()
        {
            FarmCameraFit.ApplyToMainCamera();
            // B655 AIGC label duty (FE-20260924-B-11): the eleven TJ AI-assisted art
            // pieces plus the wired AI BGM flip the explicit AI-content label rules on.
            // The footer builds AFTER the camera fit so the camera-relative placement
            // lands on the fitted design frame (root-level: the shell never sees it).
            FarmAbout.EnsureAigcLabel();
        }
    }
}
// FarmCards.cs
// FarmCards.cs - the crop/livestock card system (G13-T006, SYS03).
// Ten cards on one closed roster: the six fictional crops from the frozen
// table plus the four livestock rows. Draws are a pure rotation (zero RNG:
// the same pull count always yields the same card order), acquisitions are
// rank-gated by the frozen unlock rows (a card the rank has not met stays
// locked - promotion is the ticket to the rest of the album), and every
// completed album volume books the frozen +2% income bonus. The livestock
// passives resolve from the frozen closed set: the hen meeting trickles
// egg income into the passive line, the old ox speeds every grow countdown,
// the guard dog contributes its frozen prevent percent to the anti-theft
// face and the pig arms the cabbage-guard comedy (the T007 event spine
// consumes these two faces). The ten-pull rides the AdCap gacha category
// with the G08 T009 three-piece law: the HasCategoryBudget precheck and the
// consumption draw the same policy instance and the same day bucket (they
// can never disagree), the evt_ad funnel books offer -&gt; complete -&gt; landed
// in order, and a capped tap refuses BEFORE the ad ever plays.
using System;
using System.Collections.Generic;
using Sy.Platform;
namespace Sy.CrazyFarm
{
    public enum FarmCardKind
    {
        Crop,
        Livestock,
    }
    /// &lt;summary&gt;One roster card: the compiled mirror of one frozen crop or
    /// livestock row (ids and unlock ranks come from FarmData, never restated
    /// here - a schema drift shows up as a check mismatch, not a copy).&lt;/summary&gt;
    public sealed class FarmCardDef
    {
        public string id = "";
        public FarmCardKind kind;
        public int unlockRank;      // crops = 0 (the seed economy gates them), livestock = the frozen row
        public string passiveId = "";   // livestock only, the frozen closed set
```

```
    public string albumId = "";
}
/// &lt;summary&gt;One ten-pull result: the rotation order, the first-time
/// acquisitions, and the capped-refuse face (nothing played).&lt;/summary&gt;
public sealed class FarmCardDraw
{
    public string[] PulledIds = Array.Empty&lt;string&gt;();
    public string[] NewIds = Array.Empty&lt;string&gt;();
    public bool CappedRefused;
}
public sealed class FarmCards
{
    /// &lt;summary&gt;Ten-pull ad point (T006): draws from the 10/day g13_gacha
    /// category - one rewarded ad per ten-pull (AdCategories authority,
    /// g13_ prefix keeps id namespaces isolated).&lt;/summary&gt;
    public const string TenPullPointId = "g13_ten_pull";
    // ---- v1 engine wiring (sweep-tunable; frozen economy laws stay in
    // FarmData - these carry only the card feel) ----
    public const int TenPullSize = 10;              // the roster is exactly the closed card set
    public const double EggIncomeHcPerSec = 5.0;    // the hen meeting: eggs trickle into the passive line
    public const int OxGrowthBoostPct = 15;         // the old ox: plowing speeds every countdown
    readonly FarmGrowth _growth;
    readonly AdCapPolicy _adCap;       // the shared policy instance (the three-piece law)
    readonly Func&lt;string&gt; _dayKey;     // injected for deterministic gates; production rides TodayKey
    readonly List&lt;string&gt; _owned = new List&lt;string&gt;();
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _drawCursor;                    // zero RNG: the pull order is a fixed rotation
    int _tenPullsDone;
    public FarmCards(FarmGrowth growth, AdCapPolicy adCap, Func&lt;string&gt; dayKey = null)
    {
        _growth = growth;
        _adCap = adCap;
        _dayKey = dayKey ?? AdCapPolicy.TodayKey;
    }
    /// &lt;summary&gt;The closed roster: the six crops then the four livestock,
    /// ids and unlock rows read live from the frozen tables.&lt;/summary&gt;
    public static FarmCardDef[] Roster()
    {
        var list = new List&lt;FarmCardDef&gt;();
        foreach (var c in FarmData.Crops())
            list.Add(new FarmCardDef { id = c.id, kind = FarmCardKind.Crop, unlockRank = 0, albumId = c.albumId });
        foreach (var l in FarmData.Livestock())
            list.Add(new FarmCardDef { id = l.id, kind = FarmCardKind.Livestock, unlockRank = l.unlockRank, passiveI
        return list.ToArray();
    }
    // ---- live faces ----
    public FarmGrowth GrowthOf =&gt; _growth;
    public IReadOnlyList&lt;string&gt; Owned =&gt; _owned;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    public int TenPullsDone =&gt; _tenPullsDone;
```

```
public int DrawCursor =&gt; _drawCursor;
public bool Owns(string cardId) =&gt; _owned.Contains(cardId);
/// &lt;summary&gt;True when the rank has met the card's frozen unlock row
/// (crops are always met - the seed economy gates them).&lt;/summary&gt;
public bool UnlockMet(FarmCardDef card)
    =&gt; card.unlockRank &lt;= _growth.Rank.rank;
// ---- the album faces ----
public static string[] AlbumVolumeIds()
    =&gt; new[] { "album_crop", "album_livestock", "album_thief" };
public int VolumeSize(string albumId)
{
    int n = 0;
    foreach (var c in Roster()) if (c.albumId == albumId) n++;
    // the thief volume collects through the event spine (T007): its
    // four rows count toward the volume size, none are drawable yet.
    if (albumId == "album_thief") n = FarmData.ThiefCount;
    return n;
}
public int OwnedInVolume(string albumId)
{
    int n = 0;
    foreach (var id in _owned) if (AlbumOf(id) == albumId) n++;
    return n;
}
public bool VolumeComplete(string albumId)
    =&gt; OwnedInVolume(albumId) &gt;= VolumeSize(albumId);
/// &lt;summary&gt;Completed volumes count (the +2% law rides
/// FarmData.AlbumBonusPerVolumePct).&lt;/summary&gt;
public int CompletedVolumes()
{
    int n = 0;
    foreach (var a in AlbumVolumeIds()) if (VolumeComplete(a)) n++;
    return n;
}
public int AlbumIncomeBonusPct =&gt; CompletedVolumes() * FarmData.AlbumBonusPerVolumePct;
static string AlbumOf(string cardId)
{
    foreach (var c in Roster()) if (c.id == cardId) return c.albumId;
    return "";
}
// ---- the AdCap three-piece (G08 T009 law) ----
/// &lt;summary&gt;Live precheck: always-fresh budget face on the same policy
/// instance and the same day bucket the consumption draws from.&lt;/summary&gt;
public bool TenPullBudgetOk
    =&gt; _adCap != null &amp;&amp; _adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(TenPullPoin
/// &lt;summary&gt;The ten-pull: precheck first (a capped tap refuses BEFORE
/// the ad ever plays and books the capped funnel line), consume on the
/// same policy instance and day bucket, then resolve ten cards on the
/// zero-RNG rotation with the rank gate and book the funnel in order.
/// A null policy refuses nothing (deterministic gate face).&lt;/summary&gt;
```

```
public FarmCardDraw TryTenPull()
{
    var draw = new FarmCardDraw();
    if (_adCap != null)
    {
        if (!_adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(TenPullPointId)))
        {
            draw.CappedRefused = true;
            _funnel.Add("evt_ad_offer_capped@" + Stamp());
            return draw;   // the honest face refuses at the tap, never after the ad
        }
        if (!_adCap.TryConsumeRewarded(_dayKey(), AdCategories.CategoryOfPoint(TenPullPointId)))
        {
            draw.CappedRefused = true;
            _funnel.Add("evt_ad_offer_capped@" + Stamp());
            return draw;
        }
    }
    _tenPullsDone++;
    _funnel.Add("evt_ad_offer@" + Stamp());
    _funnel.Add("evt_ad_complete@" + Stamp());
    var roster = Roster();
    var pulled = new List&lt;string&gt;();
    var fresh = new List&lt;string&gt;();
    for (int i = 0; i &lt; TenPullSize; i++)
    {
        var card = roster[_drawCursor % roster.Length];
        _drawCursor++;
        pulled.Add(card.id);
        if (UnlockMet(card) &amp;&amp; !_owned.Contains(card.id))
        {
            _owned.Add(card.id);
            fresh.Add(card.id);
        }
    }
    draw.PulledIds = pulled.ToArray();
    draw.NewIds = fresh.ToArray();
    ApplyPassives();
    _funnel.Add("evt_gacha_landed@" + Stamp());
    return draw;
}
// ---- the passive resolution (the frozen closed set) ----
/// &lt;summary&gt;Re-resolves every owned passive against the current rank
/// and pushes the factors into the growth engine. comedy_guard and
/// theft_prevent_pct resolve on their own query faces below (the T007
/// event spine consumes them).&lt;/summary&gt;
public void ApplyPassives()
{
    bool hasHen = false, hasOx = false;
    foreach (var l in FarmData.Livestock())
```

```
            {
                if (_owned.Contains(l.id) &amp;amp;&amp;amp; l.unlockRank &amp;lt;= _growth.Rank.rank)
                {
                    if (l.passiveId == "egg_income") hasHen = true;
                    if (l.passiveId == "growth_boost_pct") hasOx = true;
                }
            }
            _growth.SetCardFactors(AlbumIncomeBonusPct, hasOx ? OxGrowthBoostPct : 0, hasHen ? EggIncomeHcPerSec : 0);
        }
        /// &amp;lt;summary&amp;gt;The guard-dog face: the frozen prevent percent of the
        /// guard_dog row when the dog card is owned (0 otherwise).&amp;lt;/summary&amp;gt;
        public int GuardPreventPct
        {
            get
            {
                if (!_owned.Contains("livestock_guard_dog")) return 0;
                foreach (var g in FarmData.Guards())
                    if (g.id == "guard_dog") return g.preventPct;
                return 0;
            }
        }
        /// &amp;lt;summary&amp;gt;The pig face: the cabbage-guard comedy stands armed once
        /// the pig card is owned (the E01 star, T007 consumes the face).&amp;lt;/summary&amp;gt;
        public bool PigGuardArmed =&amp;gt; _owned.Contains("livestock_pig_strongwill");
        string Stamp() =&amp;gt; _growth.SimTime.ToString("0.00");
        /// &amp;lt;summary&amp;gt;Canonical fingerprint: the zero-RNG parity instrument
        /// (the same script = the same bytes).&amp;lt;/summary&amp;gt;
        public string Fingerprint()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("cursor=").Append(_drawCursor);
            sb.Append("|pulls=").Append(_tenPullsDone);
            sb.Append("|owned=");
            foreach (var id in _owned) sb.Append(id).Append(',');
            sb.Append("|vols=").Append(CompletedVolumes());
            sb.Append("|bonus=").Append(AlbumIncomeBonusPct);
            sb.Append("|dog=").Append(GuardPreventPct);
            sb.Append("|pig=").Append(PigGuardArmed ? 1 : 0);
            sb.Append("|funnel=").Append(_funnel.Count);
            return sb.ToString();
        }
    }
}
// FarmData.cs
// FarmData.cs - frozen data schemas and tables for Crazy Farm (G13-T004).
// T007a added the village event table (E01-E10) - a structural move, so
// the schema version bumped 1 -&amp;gt; 2 with it.
// The numeric pack doc (G13 v1, the G13_NumPack doc face) is the source of truth and
// this file is its compiled mirror: schema freeze means the structural laws
// live here as code and DataSchemaCheck proves every law on every row.
```

```
// Values are sweep-tunable later (bot sweeps re-freeze revisions through the
// doc pack), the structure only moves with a schema version bump. Zero
// server faces: the tables ship in the build (red line 12). Compliance
// faces ride the freeze: every crop and animal stays fictional, theft is
// an NPC comedy with a local report card, and the farm disclaimer keeps
// its permanent seat. Recorded deviation from the G09 line: offline growth
// runs at full pace (the idle-genre natural face, doc M09 "crops keep
// growing").
namespace Sy.CrazyFarm
{
    /// &lt;summary&gt;One crop row: the growth economy face (M01) frozen per
    /// fictional crop. Sell price must clear seed cost x4 (the healthy
    /// margin law - an idle farm never runs a loss).&lt;/summary&gt;
    public sealed class CropDef
    {
        public string id = "";
        public int growSeconds;    // seed to mature, three even stages
        public int seedCost;       // gold per seed
        public int sellHc;         // harvest sale in gold hundredths
        public string albumId = "";
    }
    /// &lt;summary&gt;One farm rank row: the plot economy (M02) and the income
    /// ceiling frozen on one row per rank R1-R5.&lt;/summary&gt;
    public sealed class RankRow
    {
        public int rank;           // 1-5 (R1 one-mu plot .. R5 agri empire)
        public int plots;          // preset plot slots
        public int firstPlotCost; // gold cost of the first plot
        public int plotOutputHc;   // baseline plot income in gold hundredths per second
        public int incomeCapHc;    // full-rank income ceiling in gold hundredths per second
    }
    /// &lt;summary&gt;One harvest-review gate (M04, dual track): the hard metrics
    /// AND the festival review event together - a promotion is a new
    /// gameplay ticket, never a wall. Failing a review only arms the
    /// replant comedy (the village elder helps you top up).&lt;/summary&gt;
    public sealed class ReviewGateDef
    {
        public int fromRank;
        public int toRank;
        public int minPlots;
        public int minAlbum;
        public int minFame;
        public string reviewEventId = "";
    }
    /// &lt;summary&gt;One fame band row (M06, the second numeric line, 0-100
    /// segments with a visible two-way comedy face per band).&lt;/summary&gt;
    public sealed class FameBand
    {
        public int low;         // inclusive
        public int high;        // inclusive
```

```
    public string id = "";
}
/// &lt;summary&gt;One livestock row (M03): fictional animals only, each with
/// one passive from the closed set.&lt;/summary&gt;
public sealed class LivestockDef
{
    public string id = "";
    public int unlockRank;
    public string passiveId = "";   // comedy_guard / theft_prevent_pct / egg_income / growth_boost_pct
    public string albumId = "";
}
/// &lt;summary&gt;One thief row (M05, the event spine): NPC thieves collected
/// in the third album volume - the QQ-farm collective memory face.&lt;/summary&gt;
public sealed class ThiefDef
{
    public string id = "";
    public int minRank;      // the rank the thief starts raiding at
    public string albumId = "";
}
/// &lt;summary&gt;One anti-theft setup row (M05): the pick-what closed set -
/// scarecrow, guard dog or the boss personally staking out.&lt;/summary&gt;
public sealed class GuardDef
{
    public string id = "";
    public int preventPct;   // catch-chance percent when the thief strikes
}
/// &lt;summary&gt;One village event card row (T007, the comedy spine): the
/// farm cast drops scene cards - theft rows mirror the thief table
/// rank-for-rank (zero drift), comedy rows swing fame, the festival
/// row mirrors the frozen season-wheel constant, and chain rows link
/// into their follow-up card (night_intel / sack_return).&lt;/summary&gt;
public sealed class EventDef
{
    public string id = "";
    public int rank;           // 0 = any rank (resident), 1-5 = from-rank floor
    public string type = "";   // comedy / theft / festival / positive (closed set)
    public int fameSwing;      // 0 or +/-1..2 (the card's fame face)
    public string thiefId = "";// non-empty on theft rows: mirrors ThiefTable
    public string chain = "";  // "" / night_intel / sack_return (closed set)
}
public static class FarmData
{
    public const int SchemaVersion = 2;
    public const int RankCount = 5;
    public const int GateCount = 4;      // R1-&gt;R2 .. R4-&gt;R5
    public const int BandCount = 4;      // the fame 0-100 partition
    public const int CropCount = 6;
    public const int LivestockCount = 4;
    public const int ThiefCount = 4;
    public const int GuardCount = 3;
```

```
public const int EventCount = 10;     // the village event wheel (T007a)
public const int FameMin = 0;
public const int FameMax = 100;
public const int AlbumVolumeCount = 3;   // crops / livestock / thieves
// ---- section 1: the growth economy. The first crop (love-heart
// radish) matures in 6s under the &lt;=8s first-harvest anchor (red
// line 11 pacing source). ----
public const int FirstSeedMaxMs = 1500;        // first seed in soil &lt;= 1.5s
public const int FirstHarvestMaxSeconds = 8;   // first harvest lands &lt;= 8s
public const int FirstEventCardMaxSeconds = 10;// first event card (pig vs cabbage) &lt;= 10s
public const int P1FirstPlotSeconds = 60;      // P1: first plot + first sowing
public const int P50FlockSeconds = 300;        // P50: first hen + first thief raid
public const int AdsPerDayMin = 3;            // north-star ad expectation
// ---- section 4: the market price law - the sale price is a fame
// function. factorBp = 4000 + fame * 120, so fame 50 = the par
// band at exactly 1.0; high fame lines the buyers up, low fame
// leaves the crops unsold (a two-way visible face). ----
public const int MarketFactorBaseBp = 4000;
public const int MarketFactorFameBp = 120;
public static int MarketPriceAt(int sellHc, int fame)
{
    int f = ClampFame(fame);
    return sellHc * (MarketFactorBaseBp + f * MarketFactorFameBp) / 10000;
}
static int ClampFame(int v)
    =&gt; v &lt; FameMin ? FameMin : (v &gt; FameMax ? FameMax : v);
// ---- section 2: the plot curve laws ----
public const bool PresetSlotsOnly = true;     // pick WHAT opens, never WHERE (no coordinates in the schema)
public const int SlotCostLinearLaw = 1;      // n-th plot cost = firstPlotCost * n (linear, never exponential)
public const int UpgradeOutputPercentPerLevel = 150;  // greenhouse output *1.5 per level
public const int UpgradeCostPercentPerLevel = 200;    // upgrade cost *2 per level
public const int UpgradeMaxLevel = 3;                // the upgrade ladder caps at 3
public const int GreenhouseUnlockRank = 3;           // R3+ speed facility
public const int GreenhouseGrowthFactorBp = 15000;    // greenhouse growth *1.5
public static int PlotCost(RankRow row, int plotIndex)
    =&gt; row.firstPlotCost * plotIndex * SlotCostLinearLaw;   // 1-based plot index
// ---- section 3: the review-gate comedy law (never a wall) ----
public const bool ReplantComedyOnFail = true;   // the village elder tops up your display plot
// ---- section 4: the fame modifier closed set (v1) ----
public const int ReviewFameSwing = 3;      // harvest-review settlement swings +/-3
public const int EventFameSwingMin = 1;    // comedy events swing +/-1..2
public const int EventFameSwingMax = 2;
public const bool FameAutoDecay = false;   // v1: no automatic decay
// ---- section 5: the album collection bonus ----
public const int AlbumBonusPerVolumePct = 2;    // each completed volume = income +2% (v1 proposal)
// ---- section 6: the theft laws (NPC comedy, local report) ----
public const int TheftNightFactorBp = 20000;               // night raids strike twice as often
public const int TheftBaseIntervalSeconds = 300;           // R1 base strike interval
public const int TheftIntervalShrinkPerRankSeconds = 30;   // busier farms, busier thieves
public const int TheftIntervalMinSeconds = 120;            // the floor
```

```
public const int TheftLossCapPercent = 10;                // one raid never takes more than 10% (never bankrupt
public const int PreventedFameSwing = 2;                  // catching a thief red-handed: fame +2
public const bool TheftReportAlwaysGenerated = true;      // the brag-card report rides every raid (zero server
public static int TheftIntervalSeconds(int rank)
{
    int v = TheftBaseIntervalSeconds - (rank - 1) * TheftIntervalShrinkPerRankSeconds;
    return v &lt; TheftIntervalMinSeconds ? TheftIntervalMinSeconds : v;
}
// ---- section 7: the season wheel (preset scene swaps, no dynamic weather) ----
public const int SeasonCount = 4;
public const int SeasonLengthGameDays = 7;     // one season = one in-game week
public const int FestivalYieldFactorBp = 20000; // harvest festival day: output x2
public const string FestivalEventId = "evt_autumn_harvest_fest";
public static readonly string[] SeasonIds =
    { "spring_blossom", "summer_lotus", "autumn_harvest", "winter_snow" };
// ---- section 8: offline (SYS09 - crops keep growing) ----
public const int OfflineGrowthPercent = 100;       // offline growth runs at full pace (doc M09, recorded deviat
public const int OfflineCapHours = 12;            // per-absence cap
public const string OfflineDoubleCategory = "double"; // AdCap family category
public const int DoubleOfferPerSpan = 1;          // one double offer per absence span
public const int DoubleWinMinPer200 = 1;          // frequency window 1-5 per 200 (family law)
public const int DoubleWinMaxPer200 = 5;
// ---- section 10: the AdCap category closed set ----
public const string GachaCategory = "gacha";      // ten-pull crop/livestock draws
public const int GachaWinMinPer200 = 1;
public const int GachaWinMaxPer200 = 5;
public const string BoostCategory = "boost";      // festival-window growth boosts
public const int BoostWinMinPer200 = 1;
public const int BoostWinMaxPer200 = 5;
// ---- section 8: second-run inheritance (M10, the empire ending) ----
public const int InheritGoldPercent = 20;        // floor
public const bool InheritAlbumsKept = true;     // the three albums persist
public const bool InheritVillagerTies = true;   // the village ties follow the new run
public const bool InheritRankReset = true;     // the spine restarts at the one-mu plot
/// &lt;summary&gt;Rank names (English telemetry face; the zh copy rides the texts table).&lt;/summary&gt;
public static readonly string[] RankNames =
    { "ONE-MU PLOT", "FAMILY FARM", "MECHANIZED FARM", "ECO ESTATE", "AGRI EMPIRE" };
// ---- the crop table: growth economy (healthy margins, fast first radish) ----
static readonly CropDef[] CropTable =
{
    new CropDef { id = "crop_love_radish",        growSeconds =   6, seedCost =   5, sellHc =   40, albumId = "a
    new CropDef { id = "crop_crooked_corn",        growSeconds =  30, seedCost =  20, sellHc =  150, albumId = "
    new CropDef { id = "crop_shy_mushroom",       growSeconds =  45, seedCost =  40, sellHc =  320, albumId = "a
    new CropDef { id = "crop_twin_watermelon",    growSeconds =  60, seedCost =  60, sellHc =  450, albumId = "a
    new CropDef { id = "crop_snoring_pumpkin",    growSeconds =  90, seedCost = 100, sellHc =  800, albumId = "a
    new CropDef { id = "crop_introvert_sunflower", growSeconds = 120, seedCost = 160, sellHc = 1300, albumId = "
};
public static CropDef[] Crops() =&gt; (CropDef[])CropTable.Clone();
// ---- the rank table: plot economy + income ceiling on one row ----
static readonly RankRow[] RankTable =
```

```
{
    new RankRow { rank = 1, plots =  1, firstPlotCost =   50, plotOutputHc =  40, incomeCapHc =  120 },
    new RankRow { rank = 2, plots =  3, firstPlotCost =  120, plotOutputHc =  60, incomeCapHc =  300 },
    new RankRow { rank = 3, plots =  5, firstPlotCost =  300, plotOutputHc =  90, incomeCapHc =  700 },
    new RankRow { rank = 4, plots =  8, firstPlotCost =  700, plotOutputHc = 130, incomeCapHc = 1400 },
    new RankRow { rank = 5, plots = 12, firstPlotCost = 1500, plotOutputHc = 180, incomeCapHc = 2500 },
};
public static RankRow[] Ranks() =&gt; (RankRow[])RankTable.Clone();
// ---- harvest-review gates (dual track; the festival names ride
// the texts table, the events arm the replant comedy) ----
static readonly ReviewGateDef[] GateTable =
{
    new ReviewGateDef { fromRank = 1, toRank = 2, minPlots = 1, minAlbum =  3, minFame = 30, reviewEventId = "ev
    new ReviewGateDef { fromRank = 2, toRank = 3, minPlots = 3, minAlbum =  6, minFame = 45, reviewEventId = "ev
    new ReviewGateDef { fromRank = 3, toRank = 4, minPlots = 5, minAlbum = 10, minFame = 55, reviewEventId = "ev
    new ReviewGateDef { fromRank = 4, toRank = 5, minPlots = 8, minAlbum = 14, minFame = 65, reviewEventId = "ev
};
public static ReviewGateDef[] Gates() =&gt; (ReviewGateDef[])GateTable.Clone();
// ---- fame bands (the 0-100 partition, four visible faces) ----
static readonly FameBand[] BandTable =
{
    new FameBand { low = 0,  high = 24,  id = "fame_bad" },    // unsold crops + the lazy uncle mooching dinner
    new FameBand { low = 25, high = 49,  id = "fame_plain" },  // quiet market days, business as usual
    new FameBand { low = 50, high = 74,  id = "fame_good" },   // buyer queues at the gate
    new FameBand { low = 75, high = 100, id = "fame_hot" },    // helpers showing up + festival bonus
};
public static FameBand[] Bands() =&gt; (FameBand[])BandTable.Clone();
public static FameBand BandFor(int fame)
{
    foreach (var b in BandTable)
        if (fame &gt;= b.low &amp;&amp; fame &lt;= b.high) return b;
    return BandTable[0];
}
// ---- the fictional livestock table (M03) ----
static readonly LivestockDef[] LivestockTable =
{
    new LivestockDef { id = "livestock_pig_strongwill", unlockRank = 1, passiveId = "comedy_guard",       albumI
    new LivestockDef { id = "livestock_guard_dog",      unlockRank = 1, passiveId = "theft_prevent_pct",  albumI
    new LivestockDef { id = "livestock_hen_meeting",    unlockRank = 2, passiveId = "egg_income",         albumI
    new LivestockDef { id = "livestock_old_ox",         unlockRank = 3, passiveId = "growth_boost_pct",  albumId
};
public static LivestockDef[] Livestock() =&gt; (LivestockDef[])LivestockTable.Clone();
// ---- the NPC thief table (M05, the third album volume) ----
static readonly ThiefDef[] ThiefTable =
{
    new ThiefDef { id = "thief_night_rider",   minRank = 1, albumId = "album_thief" },  // the night rider (E02
    new ThiefDef { id = "thief_sack_brothers", minRank = 2, albumId = "album_thief" },  // the sack brothers (E0
    new ThiefDef { id = "thief_wall_auntie",   minRank = 2, albumId = "album_thief" },  // the wall-climbing aun
    new ThiefDef { id = "thief_ladder_bro",   minRank = 4, albumId = "album_thief" },  // ladder bro (late game)
};
```

```
public static ThiefDef[] Thieves() =&gt; (ThiefDef[])ThiefTable.Clone();
// ---- the anti-theft setup closed set (M05 pick-one trio) ----
static readonly GuardDef[] GuardTable =
{
    new GuardDef { id = "guard_scarecrow", preventPct = 25 },
    new GuardDef { id = "guard_dog",       preventPct = 40 },
    new GuardDef { id = "guard_stakeout",  preventPct = 60 },   // the boss personally staking out - strongest,
};
public static GuardDef[] Guards() =&gt; (GuardDef[])GuardTable.Clone();
// ---- the village event table (T007a data layer, E01-E10) ----
static readonly EventDef[] EventTable =
{
    new EventDef { id = "evt_pig_cabbage",         rank = 0, type = "comedy",   fameSwing =  0, thiefId = "",
    new EventDef { id = "evt_night_rider",         rank = 1, type = "theft",    fameSwing =  0, thiefId = "thief
    new EventDef { id = "evt_sack_brothers",       rank = 2, type = "theft",    fameSwing =  0, thiefId = "thief
    new EventDef { id = "evt_wall_auntie",         rank = 2, type = "theft",    fameSwing =  0, thiefId = "thief
    new EventDef { id = "evt_lazy_uncle",          rank = 1, type = "comedy",   fameSwing = -1, thiefId = "",
    new EventDef { id = "evt_hen_meeting",         rank = 2, type = "comedy",   fameSwing =  1, thiefId = "",
    new EventDef { id = "evt_autumn_harvest_fest", rank = 0, type = "festival", fameSwing =  2, thiefId = "",
    new EventDef { id = "evt_ladder_bro",          rank = 4, type = "theft",    fameSwing =  0, thiefId = "thief
    new EventDef { id = "evt_sack_return",         rank = 2, type = "theft",    fameSwing =  0, thiefId = "thief
    new EventDef { id = "evt_agri_expo",           rank = 3, type = "festival", fameSwing =  2, thiefId = "",
};
public static EventDef[] Events() =&gt; (EventDef[])EventTable.Clone();
/// &lt;summary&gt;Canonical signature - the dual-derive byte-parity
/// instrument (the schema freeze proof): two derives of the frozen
/// tables agree here or nowhere.&lt;/summary&gt;
public static string Signature()
{
    var sb = new System.Text.StringBuilder();
    sb.Append("v").Append(SchemaVersion);
    sb.Append("|crops:");
    foreach (var c in CropTable)
        sb.Append(c.id).Append('/')
          .Append(c.growSeconds).Append(',').Append(c.seedCost).Append(',').Append(c.sellHc).Append(';');
    sb.Append("|ranks:");
    foreach (var r in RankTable)
        sb.Append(r.rank).Append('/')
          .Append(r.plots).Append(',').Append(r.firstPlotCost).Append(',')
          .Append(r.plotOutputHc).Append(',').Append(r.incomeCapHc).Append(';');
    sb.Append("|pacing:").Append(FirstSeedMaxMs).Append(',').Append(FirstHarvestMaxSeconds).Append(',')
      .Append(FirstEventCardMaxSeconds).Append(',').Append(P1FirstPlotSeconds).Append(',').Append(P50FlockSecond
    sb.Append("|market:").Append(MarketFactorBaseBp).Append('+').Append(MarketFactorFameBp);
    sb.Append("|plots:").Append(PresetSlotsOnly).Append(',').Append(SlotCostLinearLaw).Append(',')
      .Append(UpgradeOutputPercentPerLevel).Append(',').Append(UpgradeCostPercentPerLevel).Append(',').Append(Up
      .Append(GreenhouseUnlockRank).Append(',').Append(GreenhouseGrowthFactorBp);
    sb.Append("|gate:").Append(ReplantComedyOnFail);
    sb.Append("|fame:").Append(ReviewFameSwing).Append(',').Append(EventFameSwingMin).Append(',')
      .Append(EventFameSwingMax).Append(',').Append(FameAutoDecay);
    foreach (var b in BandTable) sb.Append(',').Append(b.id).Append(':').Append(b.low).Append('-').Append(b.high
```

```
            sb.Append("|album:").Append(AlbumVolumeCount).Append(',').Append(AlbumBonusPerVolumePct);
            sb.Append("|theft:").Append(TheftNightFactorBp).Append(',').Append(TheftBaseIntervalSeconds).Append(',')
              .Append(TheftIntervalShrinkPerRankSeconds).Append(',').Append(TheftIntervalMinSeconds).Append(',')
              .Append(TheftLossCapPercent).Append(',').Append(PreventedFameSwing).Append(',').Append(TheftReportAlwaysGe
            foreach (var g in GuardTable) sb.Append(',').Append(g.id).Append('=').Append(g.preventPct);
            sb.Append("|season:").Append(SeasonLengthGameDays).Append(',').Append(FestivalYieldFactorBp).Append(',').App
            foreach (var s in SeasonIds) sb.Append(',').Append(s);
            sb.Append("|offline:").Append(OfflineGrowthPercent).Append(',').Append(OfflineCapHours).Append(',')
              .Append(OfflineDoubleCategory).Append(',').Append(DoubleOfferPerSpan).Append(',')
              .Append(DoubleWinMinPer200).Append(',').Append(DoubleWinMaxPer200);
            sb.Append("|adcap:").Append(GachaCategory).Append(',').Append(GachaWinMinPer200).Append(',').Append(GachaWin
              .Append(BoostCategory).Append(',').Append(BoostWinMinPer200).Append(',').Append(BoostWinMaxPer200);
            sb.Append("|inherit:").Append(InheritGoldPercent).Append(',').Append(InheritAlbumsKept).Append(',')
              .Append(InheritVillagerTies).Append(',').Append(InheritRankReset);
            sb.Append("|gates:");
            foreach (var g in GateTable)
                sb.Append(g.fromRank).Append('&gt;').Append(g.toRank).Append('/')
                  .Append(g.minPlots).Append(',').Append(g.minAlbum).Append(',').Append(g.minFame)
                  .Append(',').Append(g.reviewEventId).Append(';');
            sb.Append("|livestock:");
            foreach (var l in LivestockTable)
                sb.Append(l.id).Append('/').Append(l.unlockRank).Append('@').Append(l.passiveId).Append(';');
            sb.Append("|thieves:");
            foreach (var t in ThiefTable) sb.Append(t.id).Append('/').Append(t.minRank).Append(';');
            sb.Append("|events:");
            foreach (var e in EventTable)
                sb.Append(e.id).Append('/').Append(e.rank).Append('@').Append(e.type)
                  .Append('~').Append(e.fameSwing).Append('+').Append(e.thiefId).Append('#').Append(e.chain).Append(';')
            return sb.ToString();
        }
    }
}
// FarmEventEngine.cs
// FarmEventEngine.cs - the village event engine (G13-T007b, SYS04).
// Series-shared SYS04 family asset re-skinned from the G08 CampusEventEngine
// paradigm: table-agnostic pacing laws, the pool cursor is a deterministic
// rotation (zero RNG), zero wall-clock, zero server faces (red line 12).
// Two clocks ride one injected sim: the card rotation drops the comedy /
// festival / positive cards (first card inside the &lt;=10s red-line window -
// the pig-vs-cabbage first laugh - then one per the 240s cadence band), and
// the theft clock walks the thief table rank for rank (first raid on the
// frozen interval anchor, then twice as often at night). Raids resolve
// through the guard accumulation law (every strike adds the guard's prevent
// percent to a running accumulator - 100 books a catch and clears it, the
// 25/40/60 rows stay exact over the long run), a catch swings fame and
// books the evt_theft_prevented line, a loot books the theft loss on its
// own ledger line (floor of the mature field value x the loss cap - never
// bankrupt), and EVERY raid generates its report card: prevented / loot /
// empty-bag, the brag-card face of the QQ-farm collective memory. Chain
// beats are pure telemetry (the card fx already carries the net effect):
```

```
// the night-intel preview rides the pig-cabbage card same-tick (the 9.5s
// three-beat), the sack-return visit trails the sack brothers' raid.
using System;
using System.Collections.Generic;
namespace Sy.CrazyFarm
{
    public enum FarmEventKind { Card, ChainStep, Raid }
    /// &lt;summary&gt;One logged event beat: a comedy/festival card, a chain step,
    /// or a thief raid (the raid is its own event card - theft table rows
    /// fire on the theft clock, never the card rotation).&lt;/summary&gt;
    public sealed class FarmEventLogEntry
    {
        public double t;
        public string id = "";
        public FarmEventKind kind;
        public string step = "";        // chain step tag, "" on the card/raid itself
        public int fameSwing;           // the card's fame face (chain steps carry zero)
        public string thiefId = "";     // non-empty on raids
    }
    /// &lt;summary&gt;One theft report card: generated on EVERY raid (the frozen
    /// always-generated law) with one of three faces - the catch, the loot,
    /// or the empty bag (the thief trips over the fence and finds nothing).&lt;/summary&gt;
    public sealed class TheftReportCard
    {
        public double t;
        public string eventId = "";
        public string thiefId = "";
        public bool prevented;
        public int lossHc;
        public string face = "";        // "prevented" / "loot" / "empty"
    }
    public sealed class FarmEventEngine
    {
        // pacing baselines (the only-up law: these may tighten, never loosen)
        public const double FirstCardDelaySeconds = FarmStage.FirstEventCardAt;  // 9.5s: pig-cabbage first laugh + nigh
        public const double CardCadenceSeconds = 240;  // later cards: one per the 3-5 min band
        public const double CadenceBandMin = 180;
        public const double CadenceBandMax = 300;
        public const double ChainStepDelay = 0.0;      // farm chains land same-tick (the preview rides the 9.5s card be
        readonly FarmGrowth _growth;
        readonly EventDef[] _table;
        readonly List&lt;FarmEventLogEntry&gt; _log = new List&lt;FarmEventLogEntry&gt;();
        readonly List&lt;ChainBeat&gt; _pending = new List&lt;ChainBeat&gt;();
        readonly List&lt;TheftReportCard&gt; _reports = new List&lt;TheftReportCard&gt;();
        readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
        int _rank;
        int _cardCursor;
        int _theftCursor;
        int _fired;
        int _raids;
```

```
{
    _stage = stage;
    _adCap = adCap;
    _ads = ads;
    // the precheck reads the very bucket the registered service consumes
    // (injected key only for deterministic gates; production rides TodayKey)
    _dayKey = dayKey ?? AdCapPolicy.TodayKey;
    if (_ads != null) _ads.OnAdEvent += OnAdLifecycle;
}
public FarmOfflineRun LastRun { get { return _run; } }
/// &lt;summary&gt;Live double-category precheck: always-fresh budget face on
/// the same policy instance and the same day bucket the registered
/// service consumes from, so precheck and consume can never disagree.&lt;/summary&gt;
public bool OfferBudgetOk
{
    get
    {
        return _adCap != null
            &amp;&amp; _adCap.HasCategoryBudget(_dayKey(), AdCategories.CategoryOfPoint(OfflineDoublePoint));
    }
}
string Stamp()
{
    return _stage == null ? "0.00" : _stage.SimTime.ToString("0.00");
}
/// &lt;summary&gt;Settles an absence: cap at 12h, count the span at the
/// frozen full pace (the recorded G09 deviation), ride the whole span
/// through the very same live loop (growth books, the day clock rolls
/// the seasons, the village engine fires), then cut the long-growth
/// summary and arm the one doubler offer - an absence that resolves
/// nothing offers nothing. Capped offers book too (the honest face
/// refuses at the tap, never after the ad).&lt;/summary&gt;
public FarmOfflineRun Settle(double absentSeconds)
{
    double capped = absentSeconds &lt; 0 ? 0 : absentSeconds;
    double capSeconds = FarmData.OfflineCapHours * 3600.0;
    if (capped &gt; capSeconds) capped = capSeconds;
    double sim = capped * FarmData.OfflineGrowthPercent / 100.0;
    var g = _stage.Growth;
    int ledgerBefore = g.Ledger.soldHc + g.Ledger.plotHc;
    int dayBefore = _stage.GameDay;
    int festBefore = _stage.Season.FestivalCards.Count;
    if (sim &gt; 0)
        _stage.Advance(sim);   // the same live loop the online face rides
    var run = new FarmOfflineRun
    {
        SpanSeconds = capped,
        SimSeconds = sim,
        DaysResolved = _stage.GameDay - dayBefore,
        SpanEarnedHc = (g.Ledger.soldHc + g.Ledger.plotHc) - ledgerBefore,
```

```
            MaturePlots = CountMature(g),
            MatureFieldHc = g.MatureFieldHc(),
            FestivalReport = _stage.Season.FestivalCards.Count &gt; festBefore ? _stage.RenderFestivalReport() : "",
            OfferArmed = sim &gt; 0 &amp;&amp; _stage.GameDay - dayBefore &gt; 0
        };
        run.Summary = capped &lt;= 0 ? FarmTexts.EnText(FarmTexts.KeyOfflineSummary) : ComposeSummary(run);
        _run = run;
        if (run.OfferArmed)
            Funnel.Add("evt_ad_offer" + (OfferBudgetOk ? "" : "_capped") + "@" + Stamp());
        return run;
    }
    static int CountMature(FarmGrowth g)
    {
        int n = 0;
        for (int i = 0; i &lt; g.PlotsTotal; i++)
            if (g.IsMature(i)) n++;
        return n;
    }
    /// &lt;summary&gt;The long-growth summary face: the header, the ripe-field
    /// panorama with its awaiting coin pile, the booked village income
    /// over the settled days, and the freshest festival war report
    /// whenever a window opened inside the span. Deterministic from the
    /// run's own fields (the freeze proof recomputes it byte-for-byte).&lt;/summary&gt;
    public static string ComposeSummary(FarmOfflineRun run)
    {
        var sb = new System.Text.StringBuilder();
        sb.Append(FarmTexts.EnText(FarmTexts.KeyOfflineSummary));
        sb.Append('\n').Append(string.Format(FarmTexts.EnText(FarmTexts.KeyOfflineField),
            run.MaturePlots, run.MatureFieldHc / 100));
        sb.Append('\n').Append(string.Format(FarmTexts.EnText(FarmTexts.KeyOfflineEarned),
            run.SpanEarnedHc / 100, run.DaysResolved));
        if (!string.IsNullOrEmpty(run.FestivalReport))
            sb.Append('\n').Append(run.FestivalReport);
        return sb.ToString();
    }
    /// &lt;summary&gt;Doubler CTA: armed guard - budget precheck - rewarded
    /// view - x2 income landing. The ad only ever plays on an armed
    /// span with budget left; the x2 lands only when the view completed
    /// and lands at most once per span. A dead tap (no run, an empty
    /// span, or an already-claimed one) burns nothing: no offer entry,
    /// no ad, no budget.&lt;/summary&gt;
    public async Task&lt;bool&gt; ClaimDouble()
    {
        if (_run == null || !_run.OfferArmed) return false;
        if (!OfferBudgetOk)
        {
            Funnel.Add("evt_ad_offer_capped@" + Stamp());
            return false;
        }
        RewardResult r = await _ads.ShowRewarded(OfflineDoublePoint);
```

```
            if (r == null || !r.Completed) return false;   // evt_ad_fail rides the lifecycle face; the offer stays arme
            _stage.Growth.BookOfflineDouble(_run.SpanEarnedHc);
            _stage.StampOfflineDouble();   // the beat lands on the shared stage bus
            Funnel.Add("evt_offline_double@" + Stamp());
            _run.DoubleOffered = true;
            _run.OfferArmed = false;
            return true;
        }
        /// &lt;summary&gt;evt_ad funnel (T010): lifecycle events of the offline
        /// doubler point land in the funnel list for gate assertions; S2
        /// promotes this face to the unified telemetry pipeline.&lt;/summary&gt;
        void OnAdLifecycle(AdLifecycle e)
        {
            if (e.PointId != OfflineDoublePoint) return;
            Funnel.Add("evt_ad_" + e.Type.ToString().ToLowerInvariant() + "@" + Stamp());
        }
        public readonly List&lt;string&gt; Funnel = new List&lt;string&gt;();
    }
}
// FarmPlaceholder.cs
// FarmPlaceholder.cs - procedural placeholder faces for Crazy Farm
// (G13-T004). Art-first iron rule (X027): when ArtRuntime holds the stem
// the real sprite wins, otherwise the face falls back to a procedural
// render - views render either way and no headless gate ever depends on
// art presence. The series accent is tomato red #FF5148 (the G13 brand
// face). The emblem family is the TOMATO BLOCK (a fictional vine mark:
// every farm face carries the tomato, never a real brand).
using System.Collections.Generic;
using Sy.Core;
using UnityEngine;
namespace Sy.CrazyFarm
{
    public static class FarmPlaceholder
    {
        /// &lt;summary&gt;Series accent: tomato red #FF5148 (design-language face).&lt;/summary&gt;
        public static readonly Color AccentTomato = new Color(255f / 255f, 81f / 255f, 72f / 255f);
        // ---- rank tints: sandy cream -&gt; sprout green -&gt; wheat gold -&gt;
        // harvest amber -&gt; empire pine (the farm warms toward the empire
        // as it ranks up) ----
        static readonly Color[] RankTints =
        {
            new Color(0.96f, 0.90f, 0.78f),   // R1 one-mu plot (sandy cream)
            new Color(0.80f, 0.92f, 0.72f),   // R2 family farm (sprout green)
            new Color(0.95f, 0.85f, 0.55f),   // R3 mechanized (wheat gold)
            new Color(0.85f, 0.70f, 0.52f),   // R4 eco estate (harvest amber)
            new Color(0.55f, 0.62f, 0.50f),   // R5 agri empire (empire pine)
        };
        // ---- stems (art ships under these keys in S2.5; today they miss -&gt; fallback) ----
        public const string FacadeStem = "g13_facade";
        public const string CropStem = "g13_crop";
```

```
public const string LivestockStem = "g13_livestock";
public const string ThiefStem = "g13_thief";
public const string EventCardStem = "g13_eventcard";
public const string PanelStem = "g13_panel";
public const string PlotStem = "g13_plot";
public const string PigCabbageStem = "g13_pig_cabbage";       // A15 (U096 round-2 fix): the E01 pig-vs-cabbage
public const string VillageNightStem = "g13_village_night";   // A14 (U096 round-2 fix): the R5 empire endgame b
static readonly Dictionary&lt;string, Sprite&gt; _cache = new Dictionary&lt;string, Sprite&gt;();
/// &lt;summary&gt;True when real art ships under the stem (S2.5 face - the
/// placeholder gates branch on this so art arrival never breaks them).&lt;/summary&gt;
public static bool ArtPresent(string stem) =&gt; ArtRuntime.Get(stem) != null;
/// &lt;summary&gt;The shipped art sprite for a stem (null on the placeholder
/// path) - editor gates compare against this without touching Sy.Core.&lt;/summary&gt;
public static Sprite ArtSprite(string stem) =&gt; ArtRuntime.Get(stem);
/// &lt;summary&gt;Farm facade tinted by rank (1-5, out-of-range folds to
/// the R1 sandy face). The growth roll face.&lt;/summary&gt;
public static Sprite Facade(int rank)
{
    int i = rank - 1;
    if (i &lt; 0 || i &gt;= RankTints.Length) i = 0;
    return Face(FacadeStem + "_" + rank, RankTints[i], 64);
}
public static Sprite Crop() =&gt; Face(CropStem, new Color(0.90f, 0.86f, 0.55f), 32);
public static Sprite Livestock() =&gt; Face(LivestockStem, new Color(0.94f, 0.80f, 0.70f), 32);
public static Sprite Thief() =&gt; Face(ThiefStem, new Color(0.45f, 0.42f, 0.55f), 32);
/// &lt;summary&gt;The E01 pig-vs-cabbage star (A15, B590 wiring): art-first
/// on g13_pig_cabbage, null-safe fall-back to the thief face (X027
/// iron rule - the generic thief face holds the stage until the pig
/// art ships, then the pig art wins the E01 selection).&lt;/summary&gt;
public static Sprite PigCabbage()
{
    Sprite art = ArtRuntime.Get(PigCabbageStem);
    if (art != null) return art;
    return Thief();
}
/// &lt;summary&gt;The R5 empire endgame village-at-night backdrop (A14,
/// B590 wiring): art-first on g13_village_night, else a deep
/// night-blue block (the empire sky stub - views render either
/// way, X027 iron rule).&lt;/summary&gt;
public static Sprite VillageNight()
    =&gt; Face(VillageNightStem, new Color(0.13f, 0.16f, 0.30f), 128);
/// &lt;summary&gt;The plot face: art-first, else a tomato emblem block
/// (the fictional vine mark of the farm).&lt;/summary&gt;
public static Sprite Plot()
{
    Sprite art = ArtRuntime.Get(PlotStem);
    if (art != null) return art;
    Sprite cached;
    if (_cache.TryGetValue(PlotStem, out cached) &amp;&amp; cached != null) return cached;
    Sprite made = ProceduralTomato(32, AccentTomato, new Color(0.97f, 0.95f, 0.86f));
```

```
    _cache[PlotStem] = made;
    return made;
}
public static Sprite EventCard() =&gt; Face(EventCardStem, new Color(0.95f, 0.90f, 0.66f), 48);
public static Sprite Panel() =&gt; Face(PanelStem, new Color(0.16f, 0.18f, 0.14f, 0.92f), 64);
/// &lt;summary&gt;Art-first with null-safe procedural fallback (X027 iron rule).&lt;/summary&gt;
public static Sprite Face(string stem, Color color, int px)
{
    Sprite art = ArtRuntime.Get(stem);
    if (art != null) return art;
    string key = stem + "#" + px;
    Sprite cached;
    if (_cache.TryGetValue(key, out cached) &amp;&amp; cached != null) return cached;
    Sprite made = ProceduralSprite(color, px);
    _cache[key] = made;
    return made;
}
/// &lt;summary&gt;Procedural color block: an uncompressed readable px*px square.&lt;/summary&gt;
public static Sprite ProceduralSprite(Color color, int px)
{
    var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
    var pixels = new Color[px * px];
    for (int i = 0; i &lt; pixels.Length; i++) pixels[i] = color;
    tex.SetPixels(pixels);
    tex.Apply(false, false);
    tex.name = "sy_procedural_" + px;
    var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
    sprite.name = tex.name;
    return sprite;
}
/// &lt;summary&gt;The tomato procedural emblem: a plus-shaped vine mark in
/// the accent color on a soft block (the fictional farm mark - a
/// pure pixel draw, stable forever for gate pixel reads).&lt;/summary&gt;
public static Sprite ProceduralTomato(int px, Color tomato, Color background)
{
    var tex = new Texture2D(px, px, TextureFormat.RGBA32, false);
    var pixels = new Color[px * px];
    int arm = px / 8;                 // arm thickness (&gt;=1 on any px &gt;= 8)
    if (arm &lt; 1) arm = 1;
    int mid = px / 2;
    for (int y = 0; y &lt; px; y++)
    {
        for (int x = 0; x &lt; px; x++)
        {
            bool inMark = (x &gt;= mid - arm &amp;&amp; x &lt;= mid + arm - 1) || (y &gt;= mid - arm &amp;&amp;
            pixels[y * px + x] = inMark ? tomato : background;
        }
    }
    tex.SetPixels(pixels);
    tex.Apply(false, false);
```

```
            tex.name = "sy_tomato_mark_" + px;
            var sprite = Sprite.Create(tex, new Rect(0, 0, px, px), new Vector2(0.5f, 0.5f), px);
            sprite.name = tex.name;
            return sprite;
        }
        /// &lt;summary&gt;The rank tint for a farm rank (1-5) - the visual
        /// progress face for checks.&lt;/summary&gt;
        public static Color TintForRank(int rank)
        {
            int i = rank - 1;
            if (i &lt; 0 || i &gt;= RankTints.Length) i = 0;
            return RankTints[i];
        }
    }
}
// FarmRankEngine.cs
// FarmRankEngine.cs - the promotion review engine (G13-T008a, SYS05/SYS06).
// The five-rank ladder rides the four frozen harvest-review gates: the hard
// metrics (plots, album entries, fame) settle together with the festival
// review event - a promotion is a new gameplay ticket, never a wall. A
// passing review settles the +3 fame swing, books evt_promote and the
// co-op stamp ceremony card (village fireworks + the harvest gong, the
// ceremony data face); a failing review settles the -3 swing, arms the
// replant comedy (ReplantComedyOnFail: the elder sneaks in with a tray of
// seedlings and tops up the display plot) and books evt_inspect. The fame
// band rides BandFor - the two-way visible face: high fame lines the
// buyers up, low fame leaves the crops unsold with the lazy uncle
// mooching dinner. Zero RNG, zero wall-clock, zero server (red line 12).
using System;
using System.Collections.Generic;
namespace Sy.CrazyFarm
{
    public enum FarmRankBeatKind { Promote, Inspect }
    /// &lt;summary&gt;One logged rank beat: the promote stamp (a review passed)
    /// or the inspect fail (the replant comedy armed).&lt;/summary&gt;
    public sealed class FarmRankBeat
    {
        public double t;
        public int fromRank;
        public int toRank;              // = fromRank on a fail (no ladder move)
        public FarmRankBeatKind kind;
        public string reviewEventId = "";// the festival review face this beat settled under
        public int fameSwing;            // +3 pass / -3 fail (the frozen review swing)
        public bool promoted;
        public string failDimension = "";// "plots" / "album" / "fame" on a fail, "" on a pass
    }
    /// &lt;summary&gt;The co-op stamp ceremony card: the promotion celebration
    /// data face - the stamped seal, the village fireworks, the harvest
    /// gong.&lt;/summary&gt;
    public sealed class FarmCeremonyCard
```

```
{
    public double t;
    public int toRank;
    public string rankName = "";        // English telemetry face (zh rides the texts table)
    public string reviewEventId = "";
    public string stampKey = "";        // the ceremony banner copy key
}
/// &lt;summary&gt;The replant comedy card: armed on a failed review - the
/// village elder tops the display plot up so the next round holds
/// (never a wall, a comedy).&lt;/summary&gt;
public sealed class FarmReplantCard
{
    public double t;
    public int fromRank;
    public string reviewEventId = "";
    public string failDimension = "";
    public string armedKey = "";        // the armed-beat copy key
}
public sealed class FarmRankEngine
{
    readonly FarmGrowth _growth;    // plots + fame live here
    readonly FarmCards _cards;      // the album dimension lives here
    readonly List&lt;FarmRankBeat&gt; _beats = new List&lt;FarmRankBeat&gt;();
    readonly List&lt;FarmCeremonyCard&gt; _ceremonies = new List&lt;FarmCeremonyCard&gt;();
    readonly List&lt;FarmReplantCard&gt; _replants = new List&lt;FarmReplantCard&gt;();
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _rank;
    bool _replantArmed;
    string _armedDimension = "";
    /// &lt;summary&gt;Series wiring face: inject the growth engine (the plots
    /// and fame holder) and the card system (the album holder).&lt;/summary&gt;
    public FarmRankEngine(FarmGrowth growth, FarmCards cards, int startRank)
    {
        _growth = growth;
        _cards = cards;
        _rank = ClampRank(startRank);
    }
    static int ClampRank(int r)
        =&gt; r &lt; 1 ? 1 : (r &gt; FarmData.RankCount ? FarmData.RankCount : r);
    // ---- live faces ----
    public int Rank =&gt; _rank;
    public bool ReplantArmed =&gt; _replantArmed;
    public string ArmedDimension =&gt; _armedDimension;
    public IList&lt;FarmRankBeat&gt; Beats =&gt; _beats;
    public IList&lt;FarmCeremonyCard&gt; Ceremonies =&gt; _ceremonies;
    public IList&lt;FarmReplantCard&gt; Replants =&gt; _replants;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    /// &lt;summary&gt;The frozen gate row for a from-rank (the GateTable
    /// mirror); null past the ladder top (R5 has no gate out).&lt;/summary&gt;
    public ReviewGateDef GateFor(int fromRank)
```

```
{
    foreach (var g in FarmData.Gates())
        if (g.fromRank == fromRank) return g;
    return null;
}
// ---- the four-gate solver (hard metrics against the live farm) ----
/// &lt;summary&gt;Album entries collected across every volume (the gate's
/// album dimension: crops + livestock + thieves).&lt;/summary&gt;
public int AlbumCollected()
{
    int n = 0;
    foreach (var v in FarmCards.AlbumVolumeIds())
        n += _cards.OwnedInVolume(v);
    return n;
}
public bool PlotsMet(ReviewGateDef gate) =&gt; _growth.OpenSlots &gt;= gate.minPlots;
public bool AlbumMet(ReviewGateDef gate) =&gt; AlbumCollected() &gt;= gate.minAlbum;
public bool FameMet(ReviewGateDef gate) =&gt; _growth.Fame &gt;= gate.minFame;
/// &lt;summary&gt;The failing dimension on a miss - "plots" / "album" /
/// "fame" in table order, "" when the gate passes.&lt;/summary&gt;
public string FailingDimension(ReviewGateDef gate)
{
    if (!PlotsMet(gate)) return "plots";
    if (!AlbumMet(gate)) return "album";
    if (!FameMet(gate)) return "fame";
    return "";
}
// ---- the review transaction (the promotion engine heart) ----
/// &lt;summary&gt;Runs one harvest review against the from-rank's frozen
/// gate: pass = the +3 settle, evt_promote, the co-op stamp
/// ceremony card and the ladder move; fail = the -3 settle, the
/// replant comedy armed and evt_inspect. Fame settles through the
/// growth engine so the 0-100 clamp law owns every swing.&lt;/summary&gt;
public FarmRankBeat Review(double t, int fromRank)
{
    var gate = GateFor(fromRank);
    if (gate == null)
        throw new InvalidOperationException("no gate from rank " + fromRank);
    string fail = FailingDimension(gate);
    bool pass = fail.Length == 0;
    var beat = new FarmRankBeat
    {
        t = t,
        fromRank = gate.fromRank,
        toRank = pass ? gate.toRank : gate.fromRank,
        kind = pass ? FarmRankBeatKind.Promote : FarmRankBeatKind.Inspect,
        reviewEventId = gate.reviewEventId,
        fameSwing = pass ? FarmData.ReviewFameSwing : -FarmData.ReviewFameSwing,
        promoted = pass,
        failDimension = fail
```

```
        };
        _growth.SetFame(_growth.Fame + beat.fameSwing);
        if (pass)
        {
            _rank = ClampRank(gate.toRank);
            _growth.PromoteTo(gate.toRank);   // the economy row moves with the ladder (the ticket lands)
            _ceremonies.Add(new FarmCeremonyCard
            {
                t = t,
                toRank = gate.toRank,
                rankName = FarmData.RankNames[gate.toRank - 1],
                reviewEventId = gate.reviewEventId,
                stampKey = FarmTexts.KeyCeremonyStamp
            });
            _funnel.Add("evt_promote@" + t.ToString("0.00"));
            _replantArmed = false;
            _armedDimension = "";
        }
        else
        {
            if (FarmData.ReplantComedyOnFail)
            {
                _replantArmed = true;
                _armedDimension = fail;
                _replants.Add(new FarmReplantCard
                {
                    t = t,
                    fromRank = gate.fromRank,
                    reviewEventId = gate.reviewEventId,
                    failDimension = fail,
                    armedKey = FarmTexts.KeyReplantArmed
                });
            }
            _funnel.Add("evt_inspect@" + t.ToString("0.00"));
        }
        _beats.Add(beat);
        return beat;
    }
    // ---- the fame band two-way face (SYS06) ----
    /// &lt;summary&gt;The live band id (the frozen 0-100 partition).&lt;/summary&gt;
    public string CurrentBandId() =&gt; FarmData.BandFor(_growth.Fame).id;
    /// &lt;summary&gt;The band's visible two-way face copy - the band id rides
    /// the texts table (fame_bad .. fame_hot), a miss degrades to the
    /// key itself, never a crash.&lt;/summary&gt;
    public string CurrentBandFace() =&gt; FarmTexts.EnText(CurrentBandId());
    /// &lt;summary&gt;The market price at the live fame - the sale face of the
    /// same band (high fame lines the buyers up, low fame unsold).&lt;/summary&gt;
    public int MarketPriceForCurrentFame(int sellHc)
        =&gt; FarmData.MarketPriceAt(sellHc, _growth.Fame);
    /// &lt;summary&gt;Clears the armed replant comedy once the elder's
```

```
        /// top-up landed (the stage layer's comedy beat consumes the arm).&lt;/summary&gt;
        public void ConsumeReplantArm()
        {
            _replantArmed = false;
            _armedDimension = "";
        }
        // ---- query faces (the check layer's deterministic probes) ----
        public int CountBeats(FarmRankBeatKind kind)
        {
            int n = 0;
            for (int i = 0; i &lt; _beats.Count; i++)
                if (_beats[i].kind == kind) n++;
            return n;
        }
        public double TimeOfBeat(FarmRankBeatKind kind)
        {
            for (int i = 0; i &lt; _beats.Count; i++)
                if (_beats[i].kind == kind) return _beats[i].t;
            return -1.0;
        }
        /// &lt;summary&gt;Deterministic fingerprint - the double-run byte-parity
        /// face (zero-RNG family): identical drives land on identical bytes.&lt;/summary&gt;
        public string RankSig()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("rank=").Append(_rank);
            sb.Append("|armed=").Append(_replantArmed ? 1 : 0).Append(':').Append(_armedDimension);
            for (int i = 0; i &lt; _beats.Count; i++)
            {
                var b = _beats[i];
                sb.Append(";").Append(b.reviewEventId).Append('~')
                  .Append(b.fromRank).Append('&gt;').Append(b.toRank)
                  .Append('@').Append(b.t.ToString("0.###"))
                  .Append(b.promoted ? "#promote" : "#inspect").Append(b.promoted ? "" : ":" + b.failDimension)
                  .Append("f").Append(b.fameSwing);
            }
            sb.Append("|cer=").Append(_ceremonies.Count)
              .Append(",rep=").Append(_replants.Count)
              .Append("|funnel=").Append(_funnel.Count);
            return sb.ToString();
        }
    }
}
// FarmSeason.cs
// FarmSeason.cs - the season wheel engine (G13-T009a, SYS07). Pure
// zero-RNG season clock: the four seasons rotate as preset scene swaps
// (never dynamic weather - a performance law), one season = one
// in-game week (the frozen SeasonLengthGameDays), and the harvest
// festival window opens on Liqiu - the opening day of autumn
// (FestivalDayOfSeason, the term that starts the season by name) -
```

```
// with the frozen x2 yield factor and the local harvest report card
// (the share hammer, always generated on-device - red line 12). The
// growth-side x2 income settle is the T009b wiring slice - zero
// FarmGrowth calls here keeps the T005/T007/T008 frozen faces
// constructively drift-free, and FarmData stays untouched (schema v2
// freeze holds: every season law rides the already-frozen section 7
// constants). Zero RNG, zero wall-clock, zero server.
using System;
using System.Collections.Generic;
namespace Sy.CrazyFarm
{
    public enum FarmSeasonBeatKind { SeasonSwap, FestivalWindow }
    /// &lt;summary&gt;One logged season beat: the preset swap into a new
    /// season, or the festival window opening on Liqiu.&lt;/summary&gt;
    public sealed class FarmSeasonBeat
    {
        public int gameDay;               // 1-based in-game day the beat landed on
        public string fromSeasonId = "";
        public string toSeasonId = "";
        public FarmSeasonBeatKind kind;
    }
    /// &lt;summary&gt;The harvest festival card: the Liqiu window's data face -
    /// the frozen event mirror, the x2 yield factor and the report-window
    /// copy key (the harvest report long image, the share hammer).&lt;/summary&gt;
    public sealed class FarmFestivalCard
    {
        public int gameDay;
        public string seasonId = "";
        public string eventId = "";      // the frozen FestivalEventId mirror
        public int yieldFactorBp;        // the frozen FestivalYieldFactorBp mirror
        public string windowKey = "";     // the window-opening copy key
    }
    public sealed class FarmSeason
    {
        /// &lt;summary&gt;Liqiu - the solar term that opens autumn, so the
        /// festival window lands on the autumn season's first day (the
        /// doc names the window after the term; the term is the season's
        /// opening day by definition).&lt;/summary&gt;
        public const int FestivalDayOfSeason = 1;
        readonly List&lt;FarmSeasonBeat&gt; _beats = new List&lt;FarmSeasonBeat&gt;();
        readonly List&lt;FarmFestivalCard&gt; _cards = new List&lt;FarmFestivalCard&gt;();
        readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
        int _gameDay;
        /// &lt;summary&gt;Series wiring face: the season clock starts on a
        /// 1-based in-game day (day 1 = the first spring day of the first
        /// year). No beats emit on construction - the wheel only logs
        /// arrivals driven by AdvanceDay (the stage layer owns the day
        /// clock in T009b).&lt;/summary&gt;
        public FarmSeason(int startGameDay = 1)
        {
```

```
    _gameDay = startGameDay &lt; 1 ? 1 : startGameDay;
}
// ---- the pure season laws (the check layer's deterministic probes) ----
/// &lt;summary&gt;The season a 1-based game day falls in (the frozen
/// rotation: one season per in-game week, cycling the four
/// presets in table order).&lt;/summary&gt;
public static string SeasonForDay(int gameDay)
{
    int d = gameDay &lt; 1 ? 1 : gameDay;
    return FarmData.SeasonIds[((d - 1) / FarmData.SeasonLengthGameDays) % FarmData.SeasonCount];
}
public static int SeasonIndexForDay(int gameDay)
{
    int d = gameDay &lt; 1 ? 1 : gameDay;
    return ((d - 1) / FarmData.SeasonLengthGameDays) % FarmData.SeasonCount;
}
/// &lt;summary&gt;The 1-based day inside the current season (1 ..
/// SeasonLengthGameDays).&lt;/summary&gt;
public static int DayOfSeason(int gameDay)
{
    int d = gameDay &lt; 1 ? 1 : gameDay;
    return (d - 1) % FarmData.SeasonLengthGameDays + 1;
}
/// &lt;summary&gt;True when the game day is the Liqiu window - the
/// autumn season's opening day (the harvest festival).&lt;/summary&gt;
public static bool FestivalActive(int gameDay)
    =&gt; SeasonForDay(gameDay) == FarmData.SeasonIds[2]   // autumn_harvest
       &amp;&amp; DayOfSeason(gameDay) == FestivalDayOfSeason;
/// &lt;summary&gt;The day's yield factor in basis points: the frozen x2
/// festival factor on the Liqiu window, the neutral 10000 every
/// other day (the income-accounting face T009b settles).&lt;/summary&gt;
public static int YieldFactorBpForDay(int gameDay)
    =&gt; FestivalActive(gameDay) ? FarmData.FestivalYieldFactorBp : 10000;
// ---- live faces ----
public int GameDay =&gt; _gameDay;
public string CurrentSeasonId =&gt; SeasonForDay(_gameDay);
public bool FestivalWindowActive =&gt; FestivalActive(_gameDay);
public int CurrentYieldFactorBp =&gt; YieldFactorBpForDay(_gameDay);
public IList&lt;FarmSeasonBeat&gt; Beats =&gt; _beats;
public IList&lt;FarmFestivalCard&gt; FestivalCards =&gt; _cards;
public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
/// &lt;summary&gt;Advances the clock one in-game day. A season change
/// logs a preset-swap beat (the scene preset face T009b consumes);
/// landing on the Liqiu window logs the festival card (the report
/// long image's data face) - both on the same day when autumn
/// opens (the swap and the festival ride one arrival).&lt;/summary&gt;
public void AdvanceDay()
{
    string fromSeason = SeasonForDay(_gameDay);
    _gameDay++;
```

```
    string toSeason = SeasonForDay(_gameDay);
    if (toSeason != fromSeason)
    {
        _beats.Add(new FarmSeasonBeat
        {
            gameDay = _gameDay,
            fromSeasonId = fromSeason,
            toSeasonId = toSeason,
            kind = FarmSeasonBeatKind.SeasonSwap
        });
        _funnel.Add("evt_season_swap@" + _gameDay);
    }
    if (FestivalActive(_gameDay))
    {
        _beats.Add(new FarmSeasonBeat
        {
            gameDay = _gameDay,
            fromSeasonId = toSeason,
            toSeasonId = toSeason,
            kind = FarmSeasonBeatKind.FestivalWindow
        });
        _cards.Add(new FarmFestivalCard
        {
            gameDay = _gameDay,
            seasonId = toSeason,
            eventId = FarmData.FestivalEventId,
            yieldFactorBp = FarmData.FestivalYieldFactorBp,
            windowKey = FarmTexts.KeyEvtAutumnHarvestFest
        });
        _funnel.Add("evt_festival_window@" + _gameDay);
    }
}
/// &lt;summary&gt;Advances the clock n in-game days (day-by-day so
/// every swap and festival arrival logs - never skips a
/// window).&lt;/summary&gt;
public void AdvanceDays(int days)
{
    if (days &lt; 0) days = 0;
    for (int i = 0; i &lt; days; i++) AdvanceDay();
}
// ---- query faces (the check layer's deterministic probes) ----
public int CountSwaps()
{
    int n = 0;
    for (int i = 0; i &lt; _beats.Count; i++)
        if (_beats[i].kind == FarmSeasonBeatKind.SeasonSwap) n++;
    return n;
}
public int CountFestivalCards() =&gt; _cards.Count;
/// &lt;summary&gt;The first festival window's game day (-1 when none
```

```
        /// opened yet).&lt;/summary&gt;
        public int FirstFestivalGameDay()
            =&gt; _cards.Count &gt; 0 ? _cards[0].gameDay : -1;
        /// &lt;summary&gt;Deterministic fingerprint - the double-run byte-parity
        /// face (zero-RNG family): identical drives land on identical
        /// bytes.&lt;/summary&gt;
        public string SeasonSig()
        {
            var sb = new System.Text.StringBuilder();
            sb.Append("day=").Append(_gameDay);
            sb.Append("|season=").Append(CurrentSeasonId);
            sb.Append("|fest=").Append(FestivalWindowActive ? 1 : 0);
            sb.Append("|factor=").Append(CurrentYieldFactorBp);
            for (int i = 0; i &lt; _beats.Count; i++)
            {
                var b = _beats[i];
                sb.Append(";").Append(b.kind == FarmSeasonBeatKind.SeasonSwap ? "swap" : "fest")
                  .Append('@').Append(b.gameDay)
                  .Append(':').Append(b.fromSeasonId).Append('&gt;').Append(b.toSeasonId);
            }
            sb.Append("|cards=");
            for (int i = 0; i &lt; _cards.Count; i++)
                sb.Append(_cards[i].gameDay).Append('+').Append(_cards[i].eventId)
                  .Append('x').Append(_cards[i].yieldFactorBp).Append(',');
            sb.Append("|funnel=").Append(_funnel.Count);
            return sb.ToString();
        }
    }
}
// FarmSignal.cs
// FarmSignal.cs - the hook-signal presentation faces (G13-T021a, S3 row 3).
// The 10-second hook law (red line 11, 06-script S0) demands one measurable
// signal from the laugh/wow/flex family; these three faces measure all
// three over the frozen engine streams (G07-T020 SignalGate paradigm):
//  - LaughSignalFace (first laugh): the pig-vs-cabbage card beat itself -
//    the frozen 9.5s E01 anchor inside the &lt;=10s red line, the tag rides
//    the card tick (same-frame, window 0), bounded by the tag life law;
//  - WowSignalFace (first wow): every loot raid (an actual loss) flips the
//    thief-at-the-door loss shot on the raid tick itself - a 0.35x slow-mo
//    window with a bounded duration and an exact unscaled restore to 1.0
//    (never a stranded slow clock); prevented and empty-bag raids never
//    engage it (the catch owns the defense report, T020 law);
//  - FlexSignalFace (first flex): every harvest-war-report share compose
//    and the empire all-lit ending each book one evt_signal_perfect funnel
//    line on the beat tick itself (same-tick), banner + CTA bounded by
//    the hold law.
// Pure read-only presentation over FarmEventEngine/FarmAlbum: zero new
// game state, zero engine changes, zero RNG, zero new text keys (the tag
// lines reuse the frozen copy surface) - the same drives always
// fingerprint byte-identical. The T021b SignalGateCheck consumes these
```

```
// faces. Audio hooks (scythe swish x3 + rooster two-note rise) defer to
// the TJ audio batch - presentation constants only here.
using System;
using System.Collections.Generic;
using System.Globalization;
namespace Sy.CrazyFarm
{
    /// &lt;summary&gt;The first-laugh face: the pig-vs-cabbage comedy card is the
    /// hook's own first laugh - the face re-derives from the event log every
    /// Note call (idempotent, zero incremental state), the tag line rides the
    /// frozen copy key, and the visibility window is bounded by the tag
    /// life law (inside the 3s perceivable promise).&lt;/summary&gt;
    public sealed class LaughSignalFace
    {
        public const string CardId = "evt_pig_cabbage";   // the E01 resident (rank-0, night_intel chain)
        public const double TagLifeSec = 2.0;             // bounded tag window, inside the 3s perceivable law
        readonly List&lt;double&gt; _beats = new List&lt;double&gt;();
        double _firstAt = -1.0;
        /// &lt;summary&gt;Re-derive from the card log: every pig-cabbage card beat
        /// at or before t counts, the first one is the first laugh.&lt;/summary&gt;
        public void Note(FarmEventEngine events, double t)
        {
            _beats.Clear();
            _firstAt = -1.0;
            for (int i = 0; i &lt; events.Log.Count; i++)
            {
                var e = events.Log[i];
                if (e.kind != FarmEventKind.Card || e.id != CardId || e.t &gt; t) continue;
                if (_beats.Count == 0) _firstAt = e.t;
                _beats.Add(e.t);
            }
        }
        public double FirstLaughAt =&gt; _firstAt;
        public int LaughBeats =&gt; _beats.Count;
        public string TagLine =&gt; FarmTexts.EnText(FarmTexts.KeyEvtPigCabbage);
        /// &lt;summary&gt;The frozen hook window law: the first laugh must land
        /// inside the &lt;=10s red line (E3 sect-7).&lt;/summary&gt;
        public bool InsideHookWindow
            =&gt; _firstAt &gt;= 0.0 &amp;&amp; _firstAt &lt;= FarmData.FirstEventCardMaxSeconds;
        /// &lt;summary&gt;The tag visibility window: alive from the beat to the
        /// beat + TagLifeSec, dark after - never a stranded tag.&lt;/summary&gt;
        public bool VisibleAt(double t)
        {
            for (int i = 0; i &lt; _beats.Count; i++)
                if (t &gt;= _beats[i] &amp;&amp; t &lt; _beats[i] + TagLifeSec) return true;
            return false;
        }
        /// &lt;summary&gt;Deterministic fingerprint (the double-run byte-parity
        /// instrument, zero-RNG family law).&lt;/summary&gt;
        public string Fingerprint()
```

```
        =&gt; "beats=" + _beats.Count + "|first=" + _firstAt.ToString("0.00")
           + "|tag=" + TagLine.Length + "|life=" + TagLifeSec.ToString("0.0");
}
/// &lt;summary&gt;The first-wow face: the thief-at-the-door loss shot. A loot
/// raid (the thief actually takes crops, lossHc &amp;gt; 0) flips the slow-mo
/// window on the raid tick itself; the window runs at the frozen 0.35x
/// scale for a bounded duration, then restores the exact unscaled 1.0 -
/// the restore law is explicit, never a stranded slow clock. Caught
/// raids own the defense report (T020) and empty-bag raids own the
/// comedy line - neither ever engages the loss shot.&lt;/summary&gt;
public sealed class WowSignalFace
{
    public const double SlowMoScale = 0.35;         // the loss-shot slow-mo window scale (frozen)
    public const double SlowMoDurationSec = 0.9;    // bounded window (&lt;= 1.0, the family law)
    public const double RestoreScale = 1.0;         // the exact unscaled restore
    readonly List&lt;double&gt; _beats = new List&lt;double&gt;();
    double _firstAt = -1.0;
    int _firstLossHc;
    /// &lt;summary&gt;Re-derive from the raid reports: every loot-face report
    /// at or before t counts, the first one anchors the loss shot.&lt;/summary&gt;
    public void Note(FarmEventEngine events, double t)
    {
        _beats.Clear();
        _firstAt = -1.0;
        _firstLossHc = 0;
        for (int i = 0; i &lt; events.Reports.Count; i++)
        {
            var r = events.Reports[i];
            if (r.face != "loot" || r.t &gt; t) continue;
            if (_beats.Count == 0) { _firstAt = r.t; _firstLossHc = r.lossHc; }
            _beats.Add(r.t);
        }
    }
    public double FirstWowAt =&gt; _firstAt;
    public int WowBeats =&gt; _beats.Count;
    public int FirstLossHc =&gt; _firstLossHc;
    /// &lt;summary&gt;The loss-shot tag line rides the frozen stolen-report
    /// copy key, the loss booked in display gold (the ComposeReportCard
    /// ledger law).&lt;/summary&gt;
    public string TagLine =&gt; _firstAt &lt; 0.0
        ? FarmTexts.EnText(FarmTexts.KeyTheftReportStolen)
        : string.Format(FarmTexts.EnText(FarmTexts.KeyTheftReportStolen), _firstLossHc / 100);
    /// &lt;summary&gt;The slow-mo law: inside a loot window the clock runs at
    /// the frozen 0.35x scale, at and after the window end the clock is
    /// exactly the unscaled 1.0 - ScaleAt(first + duration) restores.&lt;/summary&gt;
    public double ScaleAt(double t)
    {
        for (int i = 0; i &lt; _beats.Count; i++)
            if (t &gt;= _beats[i] &amp;&amp; t &lt; _beats[i] + SlowMoDurationSec) return SlowMoScale;
        return RestoreScale;
```

```
    }
    public bool SlowMoActiveAt(double t) =&gt; ScaleAt(t) == SlowMoScale;
    /// &lt;summary&gt;Deterministic fingerprint (the double-run byte-parity
    /// instrument, zero-RNG family law).&lt;/summary&gt;
    public string Fingerprint()
        =&gt; "beats=" + _beats.Count + "|first=" + _firstAt.ToString("0.00")
           + "|loss=" + _firstLossHc + "|scale=" + SlowMoScale.ToString("0.00")
           + "|dur=" + SlowMoDurationSec.ToString("0.00")
           + "|restore=" + RestoreScale.ToString("0.00");
}
/// &lt;summary&gt;The first-flex face: the harvest-war-report boast and the
/// empire all-lit ending. Every share compose (the evt_album_share_card
/// beat) and the empire ending (the panorama lights at rank R5) each
/// book one evt_signal_perfect funnel line on the beat tick itself -
/// the flex beat is always a shareable moment (red line 11, the
/// &lt;=60s shareable-moment law). The banner is the composed war report,
/// the CTA rides the frozen share-challenge copy key, both bounded by
/// the hold law.&lt;/summary&gt;
public sealed class FlexSignalFace
{
    public const string FunnelId = "evt_signal_perfect";
    public const string ShareCardBeat = "evt_album_share_card";
    public const double HoldSec = 2.0;   // bounded boast window, inside the 3s perceivable law
    readonly List&lt;string&gt; _funnel = new List&lt;string&gt;();
    int _shareBeatsSeen;
    bool _endingBooked;
    bool _endingLit;
    double _lastBeatAt = -1.0;
    string _bannerText = "";
    /// &lt;summary&gt;The booked evt_signal_perfect funnel (one line per flex
    /// beat, same-tick stamps) - the T021b gate's funnel face.&lt;/summary&gt;
    public IReadOnlyList&lt;string&gt; Funnel =&gt; _funnel;
    public int FlexBeats =&gt; _funnel.Count;
    public bool EndingLit =&gt; _endingLit;
    public double LastBeatAt =&gt; _lastBeatAt;
    public string BannerText =&gt; _bannerText;
    public string CtaLine =&gt; FarmTexts.EnText(FarmTexts.KeyShareChallenge);
    /// &lt;summary&gt;Observe the album at t: new share compose beats book on
    /// their own stamp (same-tick), the ending books once when the
    /// panorama stands armed. Idempotent per beat - a re-Note never
    /// double-books.&lt;/summary&gt;
    public void Note(FarmAlbum album, double t)
    {
        var stamps = new List&lt;double&gt;();
        for (int i = 0; i &lt; album.Funnel.Count; i++)
        {
            string f = album.Funnel[i];
            if (!f.StartsWith(ShareCardBeat + "@", StringComparison.Ordinal)) continue;
            double s = ParseStamp(f);
            if (s &lt;= t) stamps.Add(s);
```

```
            }
            while (_shareBeatsSeen &lt; stamps.Count)
            {
                double stamp = stamps[_shareBeatsSeen];
                _funnel.Add(FunnelId + "@" + stamp.ToString("0.00"));
                _shareBeatsSeen++;
                if (stamp &gt; _lastBeatAt) _lastBeatAt = stamp;
            }
            if (album.EmpireEndingArmed &amp;&amp; !_endingBooked)
            {
                _funnel.Add(FunnelId + "@" + t.ToString("0.00"));
                _endingBooked = true;
                if (t &gt; _lastBeatAt) _lastBeatAt = t;
            }
            _endingLit = album.EmpireEndingArmed;
            _bannerText = album.ComposeReportCard();
        }
        /// &lt;summary&gt;The boast visibility window: alive from the last flex
        /// beat to the beat + HoldSec, dark after - never a stranded
        /// banner.&lt;/summary&gt;
        public bool VisibleAt(double t)
            =&gt; _lastBeatAt &gt;= 0.0 &amp;&amp; t &gt;= _lastBeatAt &amp;&amp; t &lt; _lastBeatAt + HoldSec;
        static double ParseStamp(string funnelLine)
        {
            int at = funnelLine.IndexOf('@');
            if (at &lt; 0 || at + 1 &gt;= funnelLine.Length) return -1.0;
            double v;
            return double.TryParse(funnelLine.Substring(at + 1), NumberStyles.Float,
                CultureInfo.InvariantCulture, out v) ? v : -1.0;
        }
        /// &lt;summary&gt;Deterministic fingerprint (the double-run byte-parity
        /// instrument, zero-RNG family law).&lt;/summary&gt;
        public string Fingerprint()
            =&gt; "share=" + _shareBeatsSeen + "|ending=" + (_endingBooked ? 1 : 0)
               + "|lit=" + (_endingLit ? 1 : 0) + "|funnel=" + _funnel.Count
               + "|banner=" + _bannerText.Length + "|last=" + _lastBeatAt.ToString("0.00")
               + "|hold=" + HoldSec.ToString("0.0");
    }
}
// FarmStage.cs
// FarmStage.cs - the opening beat driver (G13-T005).
// Drives FarmGrowth through the frozen red-line-11 opening script and
// stamps every promise on the sim clock: the farm panorama hook owns t0,
// the first plot decision lands one second in (live right after the hook,
// zero login wall), the first seed drops with the visual+sfx+number triple
// on one tick inside the 1.5s anchor, the radish joints through three
// growth stages and the first harvest fires the coin burst triple inside
// the 8s anchor, then the pig-vs-cabbage event card (the first laugh) and
// the theft preview land inside the 10s window (T007c: the wired
// FarmEventEngine's E01 beat drives that triple - the frozen face stays
```

```
// byte-identical, OpeningSig carries no engine faces). Pure C# zero-RNG: the
// same script always fingerprints byte-identical (Redline11Check consumes
// this face).
using System.Collections.Generic;
namespace Sy.CrazyFarm
{
    public enum FarmPhase
    {
        Hook,          // the panorama + the waving-crops caption
        PlotOpen,      // the first plot decision made, soil ready
        FirstSeed,     // the first seed in the ground
        FirstHarvest,  // the first crop sold, coins burst
    }
    public sealed class FarmStage
    {
        // the frozen opening script (red-line-11 beat anchors)
        public const double FirstPlotDecisionAt = 1.0;   // T+1s: the plot decision (live right after the hook)
        public const double FirstEventCardAt = 9.5;        // T+9.5s: the pig-vs-cabbage card + the theft preview
        public const double DayLengthSeconds = 60.0;       // T009b: one in-game day per real minute (the day-clock wiri
        readonly FarmGrowth _growth;
        readonly FarmEventEngine _events;   // T007c: the village event engine rides the same sim
        readonly FarmCards _cards;          // T008b: the album dimension (the UI layer drives the pulls)
        readonly FarmRankEngine _rank;      // T008b: the promotion ladder rides the same sim
        readonly FarmSeason _season;        // T009b: the season wheel rides the stage day clock
        readonly FarmFacility _facility;    // T013b: the facility board rides the stage (zero taps = the frozen face)
        readonly FarmGuardPost _guard;      // T014: the guard-post board rides the stage (zero taps = the frozen face)
        readonly List&lt;(string id, double t)&gt; _stamps = new List&lt;(string, double)&gt;();
        double _t;
        double _dayAcc;                      // T009b: sim seconds banked toward the next in-game day
        bool _plotOpen;
        double _plotOpenedAt = -1;
        bool _seedIn;
        bool _harvestDone;
        bool _eventDone;
        string _firstEventCaption = "";
        string _signCrop = "";    // T014: the designated signature crop (the storefront treasure)
        int _seedGoldBefore;
        int _seedJumpHc;
        int _harvestGoldBefore;
        int _harvestJumpHc;
        int _seenSprout;
        int _seenBudding;
        FarmStage(FarmGrowth growth, int startRank)
        {
            _growth = growth;
            _facility = new FarmFacility();                          // T013b: fresh empty board (nothing built)
            _guard = new FarmGuardPost();                            // T014: fresh empty board (nobody on duty)
            _events = new FarmEventEngine(growth, startRank);        // T007c: the engine rides the growth sim, scoped a
            _cards = new FarmCards(growth, null);                    // T008b: fresh empty albums (the UI layer wires th
            _rank = new FarmRankEngine(growth, _cards, startRank);   // T008b: the ladder starts at the arrival rank
```

```
    _season = new FarmSeason(1);                              // T009b: the wheel opens on day 1 (the first sprin
}
/// &lt;summary&gt;The fresh farmer: an unplowed field, one preset plot
/// slot waiting, the par-band fame of a new villager.&lt;/summary&gt;
public static FarmStage NewFarmer()
    =&gt; new FarmStage(new FarmGrowth(1), 1);
/// &lt;summary&gt;The seasoned farmer (T013b): a stage arrived at rank R -
/// the staged facility faces unlock off the arrival rank the way
/// the growth row does (the wiring lane for the R3/R4/R5 build
/// menu; the fresh farmer still opens at R1).&lt;/summary&gt;
public static FarmStage AtRank(int rank)
    =&gt; new FarmStage(new FarmGrowth(rank), rank);
// ---- live faces ----
public double SimTime =&gt; _t;
public FarmPhase Phase
{
    get
    {
        if (_harvestDone) return FarmPhase.FirstHarvest;
        if (_seedIn) return FarmPhase.FirstSeed;
        if (_plotOpen) return FarmPhase.PlotOpen;
        return FarmPhase.Hook;
    }
}
public FarmGrowth Growth =&gt; _growth;
public FarmEventEngine Events =&gt; _events;   // T007c wiring face
public FarmCards Cards =&gt; _cards;           // T008b wiring face
public FarmRankEngine RankEngine =&gt; _rank;   // T008b wiring face
public FarmSeason Season =&gt; _season;          // T009b wiring face (the day-clock owner)
public FarmFacility Facility =&gt; _facility;    // T013b wiring face (the build board)
public FarmGuardPost Guard =&gt; _guard;          // T014 wiring face (the guard board)
public string CurrentSeasonId =&gt; _season.CurrentSeasonId;   // the preset swap face (S1 renders the current s
public int GameDay =&gt; _season.GameDay;
public bool ModalActive =&gt; false;              // zero login wall, zero blocking modal (red line 11)
public bool CanDecide =&gt; _plotOpen == false;  // the plot decision is live right after the hook
public IReadOnlyList&lt;(string id, double t)&gt; Stamps =&gt; _stamps;
/// &lt;summary&gt;First stamp time of an event id (-1 when never fired).&lt;/summary&gt;
public double TimeOf(string id)
{
    for (int i = 0; i &lt; _stamps.Count; i++)
        if (_stamps[i].id == id) return _stamps[i].t;
    return -1;
}
void Stamp(string id)
{
    _stamps.Add((id, _t));
}
/// &lt;summary&gt;Advances the opening clock: drives the growth economy
/// and fires the scheduled beats (first seed, growth stages, first
/// harvest, event cards) on the frozen script.&lt;/summary&gt;
```

```
public void Advance(double dt)
{
    if (dt &lt;= 0) return;
    if (TimeOf("evt_farm_hook") &lt; 0)
        Stamp("evt_farm_hook");   // the hook owns t0 (panorama + waving crops + caption)
    _t += dt;
    _growth.Advance(dt);
    _events.Advance(dt);   // T007c: the engine rides the same sim clock
    // the day clock (T009b): banked sim seconds roll the in-game
    // days, the season wheel logs its arrivals, the day's yield
    // factor lands on the growth engine and every wheel beat books
    // its stamp (the view layer swaps the scene preset off the beat
    // face - S1 renders the current season only, never dynamic
    // weather)
    _dayAcc += dt;
    while (_dayAcc &gt;= DayLengthSeconds)
    {
        _dayAcc -= DayLengthSeconds;
        int beatsBefore = _season.Beats.Count;
        _season.AdvanceDay();
        _growth.SetYieldFactorBp(_season.CurrentYieldFactorBp);
        _growth.SetBoostFactorBp(10000);   // the day clock clears the season-window boost (T015b: the window is
        for (int i = beatsBefore; i &lt; _season.Beats.Count; i++)
        {
            var beat = _season.Beats[i];
            if (beat.kind == FarmSeasonBeatKind.SeasonSwap) Stamp("evt_season_swap");
            else Stamp("evt_festival_window");
        }
    }
    // the first seed drops a fifth of a second after the plot opens
    if (_plotOpen &amp;&amp; !_seedIn &amp;&amp; _t &gt;= _plotOpenedAt + FarmGrowth.FirstSowDelaySeconds)
    {
        _seedGoldBefore = _growth.GoldHc;
        if (_growth.SowSeed(0, 0))
        {
            _seedJumpHc = _growth.GoldHc - _seedGoldBefore;   // snapshot at the stamp tick
            Stamp("evt_first_seed");
            Stamp("seed_visual");   // the seed sprite lands
            Stamp("seed_sfx");      // the signature sow chime trigger
            Stamp("seed_number");   // the gold counter tick (seed paid)
            _seedIn = true;
        }
    }
    // the jointing beats: sprout and budding crossings become visible
    if (_seedIn &amp;&amp; _growth.SproutTotal &gt; _seenSprout)
    {
        _seenSprout = _growth.SproutTotal;
        Stamp("evt_growth_sprout");
    }
    if (_seedIn &amp;&amp; _growth.BuddingTotal &gt; _seenBudding)
```

```
        {
            _seenBudding = _growth.BuddingTotal;
            Stamp("evt_growth_budding");
        }
        // the first harvest: the mature radish is tapped, the sale books
        // and the coin burst fires visual+sfx+number on one tick
        if (_seedIn &amp;&amp; !_harvestDone &amp;&amp; _growth.IsMature(0))
        {
            _harvestGoldBefore = _growth.GoldHc;
            if (_growth.TryHarvest(0))
            {
                _harvestJumpHc = _growth.GoldHc - _harvestGoldBefore;
                Stamp("evt_first_harvest");
                Stamp("coin_visual");   // the coin popup face
                Stamp("coin_sfx");     // the signature coin-burst chime
                Stamp("coin_number");  // the gold counter jump
                _harvestDone = true;
            }
        }
        // the first event cards: the wired engine's E01 beat drives the
        // stamps (the pig goes for the cabbage - the first laugh - with
        // the theft preview chained on the same tick); the stamps land on
        // the stage tick that observes the beat, the frozen T005 face
        // byte-identical (zero drift), OpeningSig carries no engine faces
        if (_plotOpen &amp;&amp; !_eventDone &amp;&amp; _events.CountCards("evt_pig_cabbage") &gt; 0)
        {
            Stamp("evt_pig_cabbage");
            Stamp("evt_first_laugh");
            Stamp("evt_theft_preview");
            _eventDone = true;
            _firstEventCaption = FarmTexts.EnText(FarmTexts.KeyEvtPigCabbage);
        }
    }
    /// &lt;summary&gt;The plot decision: opens the first preset plot slot
    /// (pick WHAT opens). Returns false while the wallet or the rank
    /// chain refuses.&lt;/summary&gt;
    public bool OpenFirstPlot()
    {
        if (_plotOpen) return false;
        if (TimeOf("evt_farm_hook") &lt; 0) return false;   // the hook owns t0
        if (!_growth.OpenNextSlot(0)) return false;
        _plotOpen = true;
        _plotOpenedAt = _t;
        Stamp("evt_plot_opened");
        Stamp("evt_first_decision");
        return true;
    }
    /// &lt;summary&gt;The harvest review beat (T008b): the player requests the
    /// review against the current rank's frozen gate. Pass books the
    /// promote stamp triple (the co-op seal + the ceremony gong) and the
```

```
/// event tiers re-scope with the ticket; fail books the inspect
/// triple with the elder's top-up landing on the same tick (the
/// replant comedy consumes its arm). Returns null before the hook or
/// past the ladder top - the review is a new-gameplay ticket, never
/// a wall.&lt;/summary&gt;
public FarmRankBeat RequestReview()
{
    if (TimeOf("evt_farm_hook") &lt; 0) return null;          // the hook owns t0
    if (_rank.GateFor(_rank.Rank) == null) return null;    // the ladder top (R5 has no gate out)
    var beat = _rank.Review(_t, _rank.Rank);
    if (beat.promoted)
    {
        Stamp("evt_promote");
        Stamp("evt_coop_stamp");        // the seal lands
        Stamp("evt_ceremony_gong");     // fireworks over the whole village + the harvest gong
        _events.SetRank(beat.toRank);  // the event tiers re-scope with the ticket
    }
    else
    {
        Stamp("evt_inspect");
        Stamp("evt_replant_comedy");    // the elder sneaks in with the seedlings tray
        Stamp("evt_elder_topup");       // the display plot gets its top-up
        if (_rank.ReplantArmed) _rank.ConsumeReplantArm();   // the comedy beat consumes the arm
    }
    return beat;
}
public int SeedJumpHc =&gt; _seedIn ? _seedJumpHc : 0;
public int HarvestJumpHc =&gt; _harvestDone ? _harvestJumpHc : 0;
public string FirstEventCaption =&gt; _firstEventCaption;
/// &lt;summary&gt;The harvest report long-image face (T009b): the Liqiu
/// window's report composes locally on-device from the freshest
/// festival card - the window copy line, the season, the game day,
/// the frozen x2 factor and the frozen event id (the share hammer,
/// always generated locally - red line 12). Empty before the first
/// window opens.&lt;/summary&gt;
public string RenderFestivalReport()
{
    var cards = _season.FestivalCards;
    if (cards.Count == 0) return "";
    var c = cards[cards.Count - 1];
    var sb = new System.Text.StringBuilder();
    sb.Append(FarmTexts.EnText(c.windowKey));
    sb.Append('|').Append(c.seasonId);
    sb.Append("|day=").Append(c.gameDay);
    sb.Append("|x").Append(c.yieldFactorBp / 10000);
    sb.Append('|').Append(c.eventId);
    return sb.ToString();
}
/// &lt;summary&gt;The facility tap (T013b): the build-menu decision lands
/// through the stage so the wallet books the rung on the same tick
```

```
    /// - the facility ledger and the wallet's facility line reconcile
    /// tick-for-tick, a refused pick never half-pays. The greenhouse
    /// rides its factors onto the growth engine the moment it stands;
    /// the panorama build debuts the estate show on the stage bus.&lt;/summary&gt;
    public bool TapFacility(int facilityIndex)
    {
        if (TimeOf("evt_farm_hook") &lt; 0) return false;   // the hook owns t0
        int goldBefore = _growth.GoldHc;
        int spentBefore = _facility.SpentHc;
        if (!_facility.BuildOrUpgrade(facilityIndex, _rank.Rank, goldBefore)) return false;
        _growth.BookFacilitySpend(_facility.SpentHc - spentBefore);   // the wallet pays the same tick
        _growth.SetFacilityFactors(_facility.GreenhouseGrowthBp, _facility.GreenhouseHarvestBp);   // the greenhouse
        Stamp("evt_facility_built");   // the menu closes, the dust settles
        if (facilityIndex == FarmFacility.Panorama)
            Stamp("evt_estate_show");   // the R5 empire show debuts on the stage bus
        return true;
    }
    /// &lt;summary&gt;The estate show render face (T013b): the panorama
    /// composition rides the stage bus - empty until the estate stands
    /// (the show waits for its promotion ticket), pure and repeatable
    /// once it does.&lt;/summary&gt;
    public string RenderEstateShow() =&gt; _facility.EstateShow(_growth);
    /// &lt;summary&gt;The guard-post tap (T014): the place-WHO decision lands
    /// through the stage so the wallet books the post on the same tick
    /// - the guard ledger and the wallet's guard line reconcile
    /// tick-for-tick, a refused pick never half-pays, and the active
    /// post's prevent percent rides the event engine's existing guard
    /// accumulator (the 25/40/60 long-run law untouched).&lt;/summary&gt;
    public bool TapGuardPost(int postIndex)
    {
        if (TimeOf("evt_farm_hook") &lt; 0) return false;   // the hook owns t0
        int goldBefore = _growth.GoldHc;
        int spentBefore = _guard.SpentHc;
        if (!_guard.Place(postIndex, _rank.Rank, goldBefore)) return false;
        _growth.BookGuardSpend(_guard.SpentHc - spentBefore);   // the wallet pays the same tick
        SetGuardPreventPct(_guard.PreventPct);                 // the engine's guard face rides live
        Stamp("evt_guard_placed");
        return true;
    }
    /// &lt;summary&gt;The signboard designation (T014, M08): the farmer picks
    /// the farm's signature crop for the storefront display - the pick
    /// only lands for a crop this farm has actually harvested (the
    /// treasure is real produce, never a poster), stamps the show beat
    /// once, and the render composes locally (the brag face, zero
    /// server - red line 12).&lt;/summary&gt;
    public bool SetSignCrop(string cropId)
    {
        if (TimeOf("evt_farm_hook") &lt; 0) return false;          // the hook owns t0
        if (string.IsNullOrEmpty(cropId) || _signCrop == cropId) return false;   // the sign already carries this cr
        if (_growth.HarvestCountOf(cropId) &lt;= 0) return false;  // never harvested here - refuse clean
```

```
    var crops = FarmData.Crops();
    bool known = false;
    foreach (var c in crops) if (c.id == cropId) { known = true; break; }
    if (!known) return false;
    _signCrop = cropId;
    Stamp("evt_sign_show");
    return true;
}
/// &lt;summary&gt;The storefront render face (T014): the signature crop's
/// live market line - empty until the farmer designates a real
/// harvest (the show waits for actual produce), pure and
/// repeatable once it does.&lt;/summary&gt;
public string RenderSignShow()
{
    if (string.IsNullOrEmpty(_signCrop)) return "";
    var crops = FarmData.Crops();
    foreach (var c in crops)
    {
        if (c.id != _signCrop) continue;
        var sb = new System.Text.StringBuilder();
        sb.Append("SIGN_SHOW|crop=").Append(c.id);
        sb.Append("|price=").Append(FarmData.MarketPriceAt(c.sellHc, _growth.Fame));
        sb.Append("|gold=").Append(_growth.GoldHc);
        return sb.ToString();
    }
    return "";
}
/// &lt;summary&gt;Offline doubler beat (T010): the completed ad view books
/// the span's whole earned income a second time; the beat rides the
/// stage stamp list so every stage-level consumer sees it (zero
/// calls keep the frozen face byte-identical).&lt;/summary&gt;
public void StampOfflineDouble() =&gt; Stamp("evt_offline_double");
/// &lt;summary&gt;The season-window boost landing (T015b): the completed
/// ad view lands the solar-term growth boost for the current game
/// day - the day clock clears it on the next roll. The beat rides
/// the stage stamp list so every stage-level consumer sees it
/// (zero calls keep the frozen face byte-identical).&lt;/summary&gt;
public bool LandBoost(int bp)
{
    if (TimeOf("evt_farm_hook") &lt; 0) return false;   // the hook owns t0
    _growth.SetBoostFactorBp(bp);
    Stamp("evt_boost_landed");
    return true;
}
/// &lt;summary&gt;Guard injection face (T007c): the cards layer wires the
/// owned guard row's prevent percent through here (zero default =
/// the unguarded T005 face).&lt;/summary&gt;
public void SetGuardPreventPct(int pct) =&gt; _events.SetGuardPreventPct(pct);
/// &lt;summary&gt;Night injection face (T007c): the day/night owner wires
/// through here (zero default = day, the T005 face).&lt;/summary&gt;
```

```
        public void SetNight(bool night) =&gt; _events.SetNight(night);
        /// &lt;summary&gt;Opening fingerprint: every stamp, the wallet and the
        /// growth state - byte-identical across identical scripts.&lt;/summary&gt;
        public string OpeningSig()
        {
            var sb = new System.Text.StringBuilder();
            foreach (var s in _stamps) sb.Append(s.id).Append('@').Append(s.t.ToString("0.###")).Append(';');
            sb.Append(_growth.Fingerprint());
            return sb.ToString();
        }
    }
}
// FarmTexts.cs
// FarmTexts.cs - the farm copy law for Crazy Farm (G13-T004).
// English source strings live here as compiled code (the family source
// law: code strings stay English); the zh-CN localization rides a separate
// data file (Assets/_Game/Configs/Texts_zhCN.json - data, not source).
// Lookup follows the shared LocaleService contract: a missing key falls
// back to the key itself (machine-readable, never a crash), and a zh
// entry missing from the data file falls back to the English source.
// Zero scattered copy: every UI-facing sentence routes through these keys
// - DataSchemaCheck scans the runtime tree for stray CJK and proves the
// coverage law. The farm disclaimer is a permanent resident key.
using System;
using System.Collections.Generic;
using Sy.Core;
namespace Sy.CrazyFarm
{
    public static class FarmTexts
    {
        // ---- key constants (the compiled copy surface) ----
        public const string KeyHookCaption = "hook_caption";
        public const string KeyFarmDisclaimer = "farm_disclaimer";   // the permanent compliance face
        public const string KeyRank1 = "rank_1";
        public const string KeyRank2 = "rank_2";
        public const string KeyRank3 = "rank_3";
        public const string KeyRank4 = "rank_4";
        public const string KeyRank5 = "rank_5";
        public const string KeyStatGold = "stat_gold";
        public const string KeyStatFame = "stat_fame";
        public const string KeyStatPlots = "stat_plots";
        public const string KeyStatAlbum = "stat_album";
        public const string KeyStatSeason = "stat_season";
        public const string KeyPlotOpen = "plot_open";
        public const string KeyPlotSlots = "plot_slots";
        public const string KeyCropSow = "crop_sow";
        public const string KeyHarvestReady = "harvest_ready";
        public const string KeyFameBad = "fame_bad";
        public const string KeyFamePlain = "fame_plain";
        public const string KeyFameGood = "fame_good";
```

```
    public const string KeyFameHot = "fame_hot";
    public const string KeyReviewVillage = "review_village";
    public const string KeyReviewTownship = "review_township";
    public const string KeyReviewCounty = "review_county";
    public const string KeyReviewExpo = "review_expo";
    public const string KeyGatePromote = "gate_promote";
    public const string KeyGateRectify = "gate_rectify";
    public const string KeyCeremonyStamp = "ceremony_stamp";   // the co-op stamp ceremony banner (T008a)
    public const string KeyReplantArmed = "replant_armed";     // the elder's top-up comedy beat (T008a)
    public const string KeyCropLoveRadish = "crop_love_radish";
    public const string KeyCropCrookedCorn = "crop_crooked_corn";
    public const string KeyCropShyMushroom = "crop_shy_mushroom";
    public const string KeyCropTwinWatermelon = "crop_twin_watermelon";
    public const string KeyCropSnoringPumpkin = "crop_snoring_pumpkin";
    public const string KeyCropIntrovertSunflower = "crop_introvert_sunflower";
    public const string KeyLivestockPigStrongwill = "livestock_pig_strongwill";
    public const string KeyLivestockGuardDog = "livestock_guard_dog";
    public const string KeyLivestockHenMeeting = "livestock_hen_meeting";
    public const string KeyLivestockOldOx = "livestock_old_ox";
    public const string KeyThiefNightRider = "thief_night_rider";
    public const string KeyThiefSackBrothers = "thief_sack_brothers";
    public const string KeyThiefWallAuntie = "thief_wall_auntie";
    public const string KeyThiefLadderBro = "thief_ladder_bro";
    public const string KeyGuardScarecrow = "guard_scarecrow";
    public const string KeyGuardDog = "guard_dog";
    public const string KeyGuardStakeout = "guard_stakeout";
    public const string KeyEvtPigCabbage = "evt_pig_cabbage";
    public const string KeyEvtNightIntel = "evt_night_intel";
    public const string KeyEvtNightRider = "evt_night_rider";
    public const string KeyEvtSackBrothers = "evt_sack_brothers";
    public const string KeyEvtWallAuntie = "evt_wall_auntie";
    public const string KeyEvtLazyUncle = "evt_lazy_uncle";
    public const string KeyEvtHenMeeting = "evt_hen_meeting";
    public const string KeyEvtAutumnHarvestFest = "evt_autumn_harvest_fest";
    public const string KeyEvtLadderBro = "evt_ladder_bro";
    public const string KeyEvtSackReturn = "evt_sack_return";
    public const string KeyEvtAgriExpo = "evt_agri_expo";
    public const string KeyTheftReportPrevented = "theft_report_prevented";
    public const string KeyTheftReportStolen = "theft_report_stolen";
    public const string KeyTheftReportEmpty = "theft_report_empty";
    public const string KeyAlbumTitle = "album_title";
    public const string KeyAlbumVolumeCrop = "album_volume_crop";
    public const string KeyAlbumVolumeLivestock = "album_volume_livestock";
    public const string KeyAlbumVolumeThief = "album_volume_thief";
    public const string KeyOfflineSummary = "offline_summary";
    public const string KeyOfflineDouble = "offline_double";
    public const string KeyOfflineField = "offline_field";     // the ripe-field panorama line (T010)
    public const string KeyOfflineEarned = "offline_earned";   // the booked village income line (T010)
    public const string KeyFestBanner = "fest_banner";
    public const string KeyTheftReport = "theft_report";
```

```
public const string KeyShareChallenge = "share_challenge";
public const string KeyDailyReport = "daily_report";
public const string KeyDrawTen = "draw_ten";
public const string KeyNewJourney = "new_journey";
public const string KeyNewJourneyConfirm = "new_journey_confirm";
public const string KeyCarryLabel = "carry_label";
public const string KeySeasonSpring = "season_spring";
public const string KeySeasonSummer = "season_summer";
public const string KeySeasonAutumn = "season_autumn";
public const string KeySeasonWinter = "season_winter";
public const string KeyReportHeader = "report_header";           // the hand-drawn war-report header (T011a)
public const string KeyReportSeason = "report_season";           // the season + game-day line (T011a)
public const string KeyReportAlbums = "report_albums";           // the three-volume count line (T011a)
public const string KeyReportGold = "report_gold";               // the gold ledger line (T011a)
public const string KeyReportSignature = "report_signature";     // the signature-crop line (T011a)
public const string KeyReportFooter = "report_footer";           // the farmer's sign-off (T011a)
public const string KeyMktHarvestBurst = "mkt_harvest_burst";    // store sheet: the harvest coin-burst beat (U0
public const string KeyMktFestLine = "mkt_fest_line";             // store sheet: the festival window beat (U096
public const string KeyMktEstateLine = "mkt_estate_line";         // store sheet: the empire finale headline (U0
/// &lt;summary&gt;English source table (compiled - the source-of-law face).&lt;/summary&gt;
public static readonly Dictionary&lt;string, string&gt; En = new Dictionary&lt;string, string&gt;
{
    { KeyHookCaption, "I JUST WANT A PLOT OF LAND - MY FARMER LIFE STARTS NOW" },
    { KeyFarmDisclaimer, "FOR ENTERTAINMENT ONLY - NOT FARMING ADVICE" },
    { KeyRank1, "ONE-MU PLOT" },
    { KeyRank2, "FAMILY FARM" },
    { KeyRank3, "MECHANIZED FARM" },
    { KeyRank4, "ECO ESTATE" },
    { KeyRank5, "AGRI EMPIRE" },
    { KeyStatGold, "GOLD" },
    { KeyStatFame, "FAME" },
    { KeyStatPlots, "PLOTS" },
    { KeyStatAlbum, "ALBUM" },
    { KeyStatSeason, "SEASON" },
    { KeyPlotOpen, "OPEN A PLOT" },
    { KeyPlotSlots, "PLOTS: {0}/{1}" },
    { KeyCropSow, "SOW THE FIRST SEED" },
    { KeyHarvestReady, "READY TO HARVEST - COINS INCOMING" },
    { KeyFameBad, "CROPS UNSOLD - THE LAZY UNCLE IS BACK FOR DINNER" },
    { KeyFamePlain, "QUIET MARKET DAYS, BUSINESS AS USUAL" },
    { KeyFameGood, "BUYER QUEUES AT THE GATE" },
    { KeyFameHot, "HELPERS SHOWING UP + FESTIVAL BONUS" },
    { KeyReviewVillage, "VILLAGE HARVEST REVIEW" },
    { KeyReviewTownship, "TOWNSHIP HARVEST REVIEW" },
    { KeyReviewCounty, "COUNTY HARVEST REVIEW" },
    { KeyReviewExpo, "AGRI EXPO GRAND REVIEW" },
    { KeyGatePromote, "PROMOTED! {0} UNLOCKED" },
    { KeyGateRectify, "THE ELDER TOPPED UP YOUR PLOT - NEXT REVIEW SOON" },
    { KeyCeremonyStamp, "THE CO-OP STAMPS THE SEAL - FIREWORKS OVER THE WHOLE VILLAGE, THE HARVEST GONG RINGS OU
    { KeyReplantArmed, "THE ELDER SNEAKS IN WITH A TRAY OF SEEDLINGS - YOUR DISPLAY PLOT GETS A TOP-UP" },
```

```
{ KeyCropLoveRadish, "LOVE-HEART RADISH" },
{ KeyCropCrookedCorn, "CROOKED-NECK CORN" },
{ KeyCropShyMushroom, "SHY MUSHROOM" },
{ KeyCropTwinWatermelon, "TWIN WATERMELON" },
{ KeyCropSnoringPumpkin, "SNORING PUMPKIN" },
{ KeyCropIntrovertSunflower, "INTROVERT SUNFLOWER" },
{ KeyLivestockPigStrongwill, "PIG STRONGWILL (CABBAGE GUARD)" },
{ KeyLivestockGuardDog, "GUARD DOG" },
{ KeyLivestockHenMeeting, "HEN MEETING" },
{ KeyLivestockOldOx, "OLD OX" },
{ KeyThiefNightRider, "NIGHT RIDER" },
{ KeyThiefSackBrothers, "SACK BROTHERS" },
{ KeyThiefWallAuntie, "WALL-CLIMBING AUNTIE" },
{ KeyThiefLadderBro, "LADDER BRO" },
{ KeyGuardScarecrow, "SCARECROW" },
{ KeyGuardDog, "GUARD DOG" },
{ KeyGuardStakeout, "STAKE IT OUT YOURSELF" },
{ KeyEvtPigCabbage, "THE PIG GOES FOR YOUR CABBAGE!" },
{ KeyEvtNightIntel, "VILLAGE INTEL: SOMEONE HAS EYES ON YOUR WATERMELON TONIGHT" },
{ KeyEvtNightRider, "THE NIGHT RIDER SLIPS OUT OF THE DARK - GUARD THE FIELD!" },
{ KeyEvtSackBrothers, "THE SACK BROTHERS DRAG THEIR SACKS IN!" },
{ KeyEvtWallAuntie, "THE AUNTIE CLIMBS YOUR WALL FOR VEGGIES AGAIN!" },
{ KeyEvtLazyUncle, "THE LAZY UNCLE IS BACK - DINNER FOR FREE AGAIN" },
{ KeyEvtHenMeeting, "THE HENS HOLD A MEETING - THE COOP IS BUZZING" },
{ KeyEvtAutumnHarvestFest, "START OF AUTUMN - THE HARVEST FEST OPENS!" },
{ KeyEvtLadderBro, "LADDER BRO LEANS A LADDER ON YOUR FENCE!" },
{ KeyEvtSackReturn, "THE SACK BROTHERS ARE BACK FOR ROUND TWO!" },
{ KeyEvtAgriExpo, "THE AGRI EXPO ROLLS INTO TOWN!" },
{ KeyTheftReportPrevented, "CAUGHT RED-HANDED - THE THIEF DROPPED THE SACK AND RAN" },
{ KeyTheftReportStolen, "THE THIEF GOT AWAY - {0} WORTH OF CROPS GONE" },
{ KeyTheftReportEmpty, "THE SACK CAME BACK EMPTY - NOTHING WAS RIPE" },
{ KeyAlbumTitle, "FARM ALBUM" },
{ KeyAlbumVolumeCrop, "CROP VOLUME" },
{ KeyAlbumVolumeLivestock, "LIVESTOCK VOLUME" },
{ KeyAlbumVolumeThief, "THIEF VOLUME" },
{ KeyOfflineSummary, "WHILE YOU WERE GONE, THE CROPS GREW TALL" },
{ KeyOfflineDouble, "DOUBLE IT?" },
{ KeyOfflineField, "THE FIELD STANDS RIPE - {0} PLOTS, {1} GOLD AWAITING THE BASKET" },
{ KeyOfflineEarned, "THE VILLAGE BOOKED {0} GOLD OVER {1} DAYS" },
{ KeyFestBanner, "AUTUMN HARVEST FEST! OUTPUT X2 TODAY" },
{ KeyTheftReport, "THEFT REPORT: {0}" },
{ KeyShareChallenge, "CAN YOUR FARM BEAT MINE?" },
{ KeyDailyReport, "DAY {0}: {1}" },
{ KeyDrawTen, "TEN-CARD SEED SPREE - THE FIELD SOWS ITSELF" },
{ KeyNewJourney, "START A NEW FARMER LIFE" },
{ KeyNewJourneyConfirm, "TAP AGAIN - THE CARRY SEEDS YOUR NEXT PLOT" },
{ KeyCarryLabel, "NEXT-FARMER CARRY: {0}" },
{ KeySeasonSpring, "SPRING BLOSSOM" },
{ KeySeasonSummer, "SUMMER LOTUS" },
{ KeySeasonAutumn, "AUTUMN HARVEST" },
```

```
            { KeySeasonWinter, "WINTER SNOW" },
            { KeyReportHeader, "HARVEST WAR REPORT" },
            { KeyReportSeason, "SEASON: {0} - GAME DAY {1}" },
            { KeyReportAlbums, "ALBUMS: CROPS {0}/{1} - LIVESTOCK {2}/{3} - THIEVES {4}/{5}" },
            { KeyReportGold, "GOLD LEDGER: {0}" },
            { KeyReportSignature, "SIGNATURE CROP: {0}" },
            { KeyReportFooter, "GLORY BELONGS TO THE FARMER" },
            { KeyMktHarvestBurst, "HARVEST! THE GOLD RAIN FALLS ON YOUR FIELD" },
            { KeyMktFestLine, "EVERY CROP SELLS AT DOUBLE PRICE TODAY" },
            { KeyMktEstateLine, "AGRI EMPIRE - THE WHOLE VILLAGE LIT" },
        };
        /// &lt;summary&gt;English lookup (missing key falls back to the key itself
        /// - machine-readable, never a crash).&lt;/summary&gt;
        public static string EnText(string key)
            =&gt; En.TryGetValue(key, out var v) &amp;&amp; !string.IsNullOrEmpty(v) ? v : key;
        /// &lt;summary&gt;Parses the zh-CN data file payload into a key-to-text
        /// map (values must be non-empty strings; anything else drops the
        /// entry so the lookup degrades to English, never to garbage).&lt;/summary&gt;
        public static Dictionary&lt;string, string&gt; ParseZh(string json)
        {
            var map = new Dictionary&lt;string, string&gt;();
            if (string.IsNullOrEmpty(json)) return map;
            try
            {
                if (!(MiniJson.Deserialize(json) is Dictionary&lt;string, object&gt; d)) return map;
                foreach (var kv in d)
                    if (!string.IsNullOrEmpty(kv.Key) &amp;&amp; kv.Value is string s &amp;&amp; !string.IsNullOrEmpty(s
                        map[kv.Key] = s;
            }
            catch
            {
                // corrupt data file = empty map = English fallback face
                return new Dictionary&lt;string, string&gt;();
            }
            return map;
        }
        /// &lt;summary&gt;LocaleService-compatible lookup delegate over a parsed
        /// zh map: zh-CN hits read the data file, everything else (and any
        /// miss) reads the English source. Wire-ready for the T005 UI face.&lt;/summary&gt;
        public static Func&lt;string, string, string&gt; LookupWithZh(Dictionary&lt;string, string&gt; zh)
        {
            return (lang, key) =&gt;
            {
                if (lang == "zh-CN" &amp;&amp; zh != null &amp;&amp; zh.TryGetValue(key, out var v) &amp;&amp; !string.I
                    return v;
                return EnText(key);
            };
        }
    }
}
```